QUANTIFI

CODEBASE REPORT

Quantifi, from the inside

What the product is, how its frontend and backend are built, how a request moves through it, the phases it went through to get here, and how it is tested.

TEAM

Krish PP
A021

Farhaan Kuka
A012

STACK

Next.js 14
TypeScript
Vercel

96,700 LINES OF CODE	**32** PAGES	**81** API ROUTES
721 COMMITS	**58** TEST SUITES	**3,700+** CHECKS, ALL PASSING

Prepared October 2026 from branch claude/landing-page-ux-0flieh at commit 50aef04.
Every figure in this report was measured from the repository itself.

Contents

PART I · OVERVIEW

1 Executive summary 4
2 What Quantifi is 5
3 The product map 7

PART II · ARCHITECTURE

4 System architecture 10
5 Runtime, deployment and scheduled work 12

PART III · FRONTEND

6 Application shell and design system 15
7 Home, discovery and markets 16
8 Stock analysis 17
9 Portfolio and watchlist 19
10 News and Financial Shorts 20
11 Community 20
12 Client infrastructure and component inventory 21

PART IV · BACKEND

13 API surface 24
14 Data acquisition and caching 27
15 Storage and data model 29
16 The analytical engine 30
17 The filings pipeline 31
18 Insider, news and AI services 33
19 Identity, access and payments 34
20 Community, analytics and administration 35
21 Security model 35

PART V · FLOWCHARTS

22 Flowcharts 38

PART VI · EVOLUTION

23 Development phases 44

PART VII · QUALITY

24 Testing approach 50
25 Test results 51
26 Test cases by domain 54

PART VIII · ASSESSMENT

27 Strengths, risks and recommendations 58
A Glossary 59
B Build prompts 61

I

PART I

Overview

What Quantifi is for, who it serves, what a visitor can do on it, and the business model the code enforces.

1 Executive summary

2 What Quantifi is

3 The product map

1 Executive summary

Quantifi is a stock research and portfolio intelligence web application built for a new generation of retail investors, with India as its home market and coverage that extends to the United States, Taiwan and the other major exchanges. It turns market prices, company filings, insider disclosures and the news into research a reader can act on: a five-axis quality score for any listed company, a valuation that is chosen to suit the kind of business being valued, portfolio analytics that measure performance against market benchmarks, a news feed rewritten as short cards, and a community where investors publish theses with the figures attached.

The codebase is a single **Next.js 14** application written in **TypeScript** and deployed on **Vercel**. There is no separate backend service: the server half of the application lives in 81 route handlers and a 31,000-line domain library, backed by **Upstash Redis** for state and **Cloudflare R2** for archived filings. Market data is fetched live from public sources; nothing on the site is demo or placeholder data.

96,700 lines of TypeScript and scripts	**118** React components	**127** domain library modules	**81** API route handlers
32 application pages	**7** scheduled data jobs	**58** test suites, all passing	**721** commits of history

What stands out in the code

- **Refusal over invention.** The single most consistent rule in the codebase is that a figure which cannot be measured is shown as unavailable, never as zero and never as an estimate presented as fact. It appears in the scoring engine, the valuation router, the filings validator, the news summariser and the currency arithmetic.
- **Industry-aware analysis.** Banks, non-bank lenders, insurers and industrial companies are read under different balance-sheet models, different income-statement layouts and different valuation methods. A bank never enters a discounted-cash-flow model.
- **Testable cores.** Every important rule lives in an importless, pure TypeScript module that a plain Node script can compile and drive with no database, network or browser. That is what makes 58 test suites possible without a test framework.
- **A security posture above its stage.** Per-request content-security nonces, encrypted user records, signed sessions, fail-closed admin access and byte-level upload sniffing are all in place.

Where the work is

The code is in good health; the open items are operational. Automated tests do not yet run in continuous integration; the pinned Next.js release has a published security advisory and should be patched; several data sources are used under terms written for personal rather than commercial use; the scheduled-job routes accept any caller if their secret is not configured; and an unused payment library and two public-prefixed configuration fallbacks should be removed. Chapter 27 sets these out with a recommended order.

HOW THIS REPORT WAS PRODUCED

Every count, file name and test result in this report was taken from the repository: the source was read, the 721-commit history was replayed to reconstruct the development phases, and all 58 test suites were run against the project's own pinned toolchain. Phases are described by what changed rather than when, as requested.

How this report is organised

PART	COVERS	READ IT FOR
I Overview	Chapters 1 to 3	What the product does, its plans and every page in it
II Architecture	Chapters 4 and 5	The system diagram, the stack, the repository layout, how a request is served and what runs on a schedule
III Frontend	Chapters 6 to 12	The design system and each user-facing surface: home, stock analysis, portfolio, news, community
IV Backend	Chapters 13 to 21	All 81 API routes, data sources, storage, the analytical engine, the filings pipeline, AI, payments and security
V Flowcharts	Chapter 22	Eight end-to-end flows drawn step by step
VI Evolution	Chapter 23	The phases the product went through, described by what changed
VII Quality	Chapters 24 to 26	How testing works, the full result of running every suite, and representative test cases
VIII Assessment	Chapter 27 and appendix	Strengths, risks, recommendations and a glossary

Readers who want the shape of the system quickly can read chapters 1, 4 and 22. Readers evaluating engineering quality should add chapters 16, 21 and 24 to 27.

2 What Quantifi is

Quantifi describes itself in its own metadata as *stock research, portfolio analysis and market theme intelligence*, and its landing page promise as AI-powered investment intelligence built for a new generation of investors. In practice it does five jobs.

JOB	WHAT THE READER GETS	WHERE IT LIVES
Research a company	A five-axis quality score, a valuation fitted to the business model, statements, peers, ownership, insider trades and news on one page	`/stock-analysis`
Track a portfolio	Multi-portfolio holdings in any currency, allocation, returns against the Nifty 50, S&P 500, FTSE 100, Nikkei 225 and Hang Seng, and an AI assistant that can read the book	`/portfolio` , `/watchlist`
Follow the market	World market summary, a heatmap of nine markets, sector returns, and a screener over the scoring engine	`/` , `/markets` , `/screener`
Read the news	A live feed matched to the companies it mentions, a written read of each story, and Financial Shorts: the day's headlines as a swipeable deck	`/news` , `/news/shorts`

JOB	WHAT THE READER GETS	WHERE IT LIVES
Discuss ideas	A community where posts carry the figures they were built from, with profiles, follows, replies, likes and published portfolios	`/community`

Who it is for

The product is pitched at younger retail investors who find ideas across social media and friends and lack a single place to check them. The design choices follow from that reader: explanations next to every score, plain-language labels (*Valuation Comfort*, *Balance Sheet Strength*) rather than analyst shorthand, rupee formatting in lakhs and crores for Indian listings, and a pricing model with a low monthly entry point.

Design principles visible in the code

PRINCIPLE	HOW THE CODE HOLDS IT
Unavailable is not zero	Every metric travels with a definition and an `unavailableReason`. The scorecard refuses to score an axis that lacks enough measured inputs rather than averaging what is left.
One yardstick per kind of business	Financial institutions route to residual-income valuation and capital-adequacy checks; industrial companies to cash-flow models and leverage, liquidity and cash-conversion checks.
A picture is a claim	The revenue Sankey conserves every flow and draws an explicit *Other* rather than padding a named bar; the heatmap sizes labels from measured glyph widths.
The summary may not exceed its source	News summaries are rejected if they contain a number that did not appear in the headline or blurb they were written from.
Privacy by construction	Published portfolios carry weights only; amounts are dropped on the device before anything is sent. Community identities are one-way hashes.
Measure before claiming	The pricing page may only claim a plan difference that a test confirms the code enforces.

Business model as enforced by the code

Two plans exist. **Explorer** is free and **Quantifi Pro** is a monthly subscription collected by Razorpay at a launch price of ₹49 (standard ₹500). The limits live in one place, `src/data/plans.ts`, and are enforced server-side so they cannot be reset by refreshing or switching device.

CAPABILITY	EXPLORER (FREE)	PRO
Full stock analyses	2 per day, counted per account per UTC day	Unlimited
News impact feed	5 stories per day	Unlimited
Watchlist	5 stocks	Unlimited
Research Ideas and playbooks	2 featured ideas	All ideas and playbooks
Insider Activity	Closed	Full access
Rare Finds and Undervalued Finds	Closed	Full access

A launch switch (`FREE_LAUNCH_OFFER`) lets a signed-in user claim a week of Pro without a card; the grant is written in the same shape as a paid one but carries an expiry, so access lapses on its own. A visitor may also read one complete analysis before being asked to sign up, because the landing page's

search opens the stock page directly.

3 The product map

The application has 32 pages under the Next.js App Router. Eight are public and indexable; the rest sit behind Google sign-in, enforced at the edge by middleware and re-checked by every data route. The table lists each page, who can reach it and what it does.

ROUTE	ACCESS	PURPOSE
`/`	PUBLIC	Landing page for visitors (an animated ascending chart carrying the product's claims); for signed-in users, the market summary, heatmap, portfolio command centre and Financial Shorts promo
`/login`	PUBLIC	Google sign-in
`/stock-analysis`	PUBLIC	The research page for any listed company or fund; the sign-up wall sits inside it, at the score
`/stocks/[ticker]`	SIGNED IN	Ticker-addressed entry into the same analysis
`/portfolio`	SIGNED IN	Multi-portfolio manager, allocation ring, benchmark comparison, AI assistant, publish-to-profile
`/watchlist`	SIGNED IN	Tracked names with score snowflakes, undervaluation, sparklines and analyst targets
`/news`	SIGNED IN	Live news matched to companies, with a written impact read per story
`/news/shorts`	SIGNED IN	Financial Shorts: one headline per screen, with likes and comments
`/community`	SIGNED IN	Feed of theses with attached evidence, composer, follows
`/community/u/[handle]`	SIGNED IN	Public profile: bio, posts, followers and any published portfolio
`/discover`	SIGNED IN	Hub for the discovery surfaces
`/ideas`, `/ideas/[slug]`	SIGNED IN PRO	Investing Ideas and themed playbooks
`/rare-finds`	PRO	Rare Finds and Undervalued Finds, scanned live through the scoring engine
`/screener`	SIGNED IN	Filter companies by score axes and presets such as quality compounders
`/markets`	SIGNED IN	World indices, sector returns and market heatmaps
`/insider-activity`	PRO	Insider buying and selling across US, Indian and Taiwanese filings, plus US legislators' trades
`/explore`	SIGNED IN	Browse companies by sector
`/famous-takes`	SIGNED IN	Well-known investor narratives
`/currencies`	SIGNED IN	Currency converter and FX tools
`/tools`, `/tools/[slug]`	SIGNED IN	Calculators, including an XIRR solver for dated cash flows
`/research/[id]`	SIGNED IN	Individual research items
`/pricing`	PUBLIC	Plan comparison and Razorpay checkout
`/billing/success`, `/billing/cancel`	SIGNED IN	Checkout return pages

ROUTE	ACCESS	PURPOSE
`/admin`	STAFF	Operations dashboard: funnel, conversions, data pipeline health and manual job runs
`/about`, `/contact`	PUBLIC	Company pages
`/privacy`, `/terms`, `/refund-policy`	PUBLIC	Legal pages

Navigation is deliberately short: *Home, News, Discover, Community, Portfolio, Watchlist* and a gold *Subscribe* link. *Discover* owns three tabs (Investing Ideas, Screener, Markets), and the Ideas tab also owns `/rare-finds`, so older links keep working while the menu stays at six items.

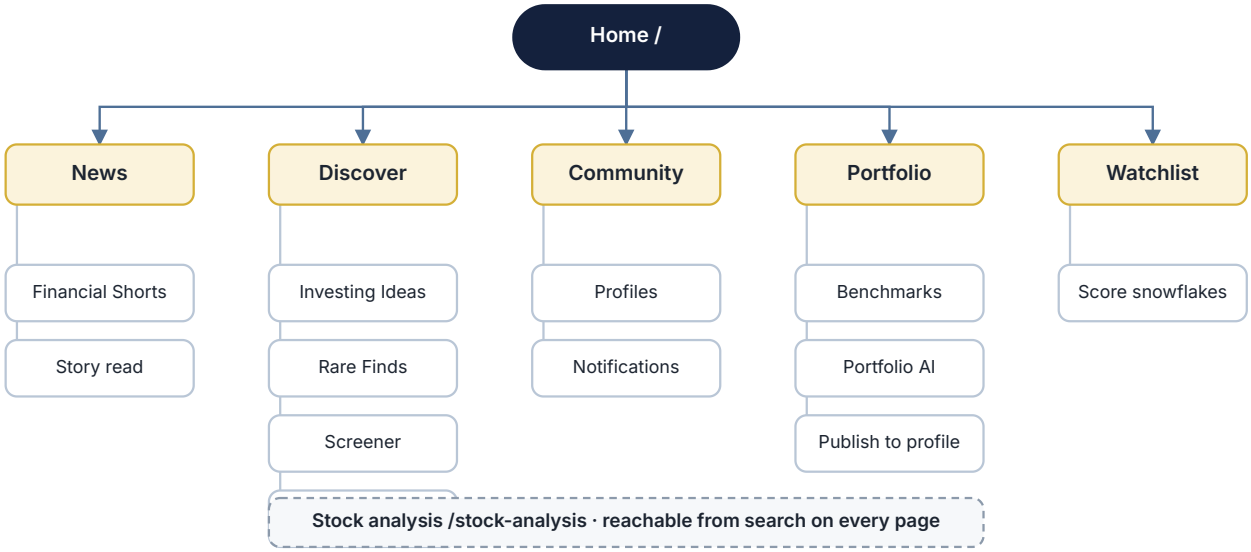


Figure 3.1 Site map. The six navigation entries and the main surfaces beneath them. The stock analysis page is reached from the command search present on every page, rather than from the menu.

II

PART II

Architecture

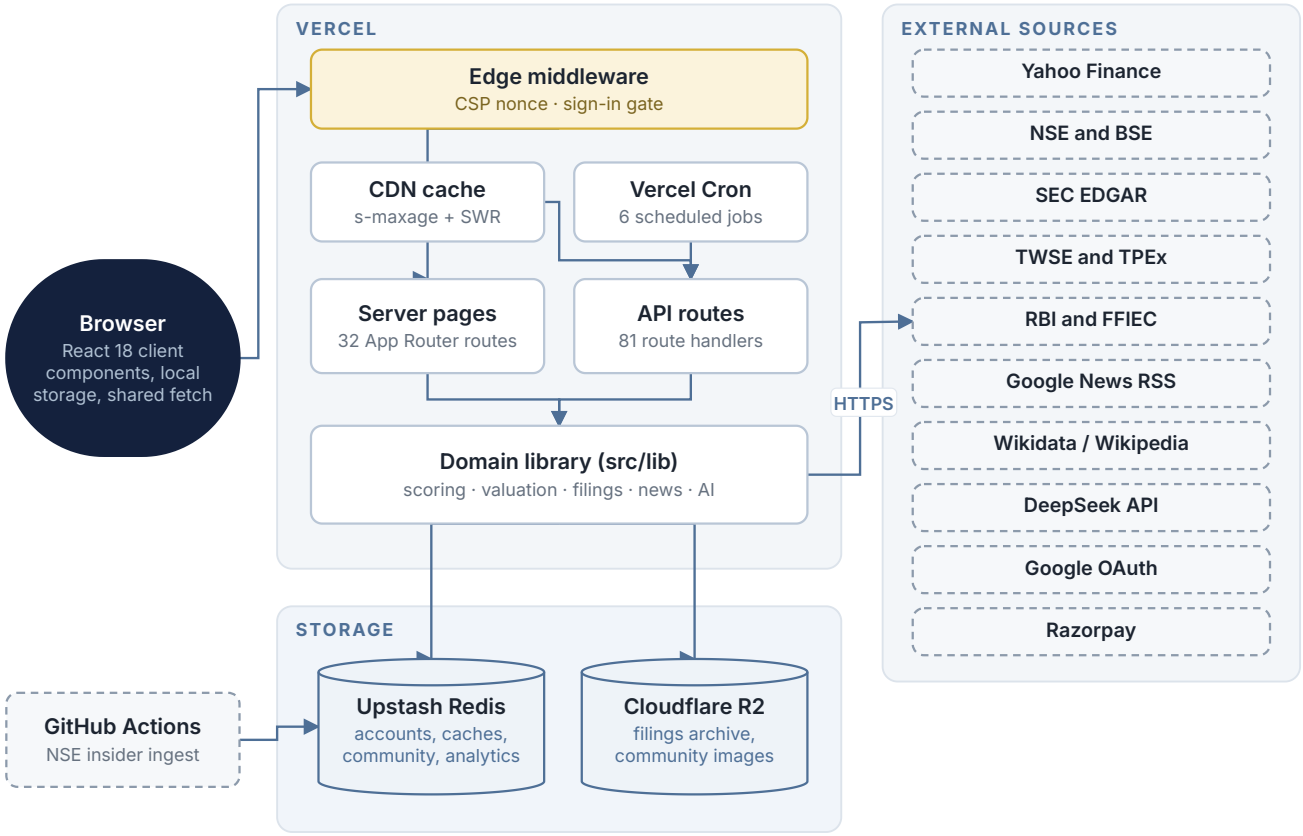
How the system is put together: the components, the services it depends on, how a request travels through it and what runs on a schedule.

4 System architecture

5 Runtime, deployment and scheduled work

4 System architecture

Quantifi is a **serverless monolith**: one Next.js codebase produces both the user interface and the API, and Vercel runs each page and route as an on-demand function. State that must outlive a request goes to Upstash Redis over its REST interface; documents that must be kept goes to Cloudflare R2. Everything else is fetched from the sources that publish it.



Arrows show the direction a request is initiated. All storage is reached over HTTPS; there is no direct database connection.

Figure 4.1 System architecture. A browser request passes the edge middleware, is served from the CDN or a function, and the function's domain library reaches storage and the external sources.

Technology stack

LAYER	CHOICE	VERSION / NOTES
Framework	Next.js App Router	14.2.15, React 18.3.1; server components for pages, route handlers for the API
Language	TypeScript	5.4.5, `strict` mode, `@/*` import alias to `src/`
Styling	Tailwind CSS	3.4.13 with a semantic colour palette and a dark-first theme
Charts	lightweight-charts, TradingView widget, hand-written SVG	Price and portfolio charts on lightweight-charts; rings, sparklines, Sankeys and treemaps drawn in SVG
HTML parsing	cheerio	Used for exchange pages, news and SEC documents
Hosting	Vercel	Serverless functions, edge middleware, CDN, Cron, Web Analytics
State	Upstash Redis	Accessed through a hand-written REST client with pipelining (`src/lib/kv.ts`)

LAYER	CHOICE	VERSION / NOTES
Archive	Cloudflare R2	S3 API with AWS Signature V4 implemented in-house (`src/lib/filings/r2.ts`)
Auth	Google OAuth 2.0	Implemented directly, with an HMAC-signed session cookie; no auth library
Payments	Razorpay	Orders, subscriptions, signature verification and webhooks over REST; no SDK
AI	DeepSeek	Chat completions plus server-side web search over its Anthropic-compatible endpoint
Fonts	Inter, IBM Plex Mono	Inter for prose and headings; Plex Mono for every figure

> **A DELIBERATE PATTERN**
>
> Where a vendor SDK would normally be installed (Redis, S3, Razorpay, authentication), the codebase talks to the REST API with `fetch` and Node's built-in `crypto` instead. The stated reasons recur in the comments: fewer dependencies that can break the build, smaller cold starts, and behaviour that can be read in one file. The production dependency list is seven packages long.

Repository layout

DIRECTORY	FILES	LINES	CONTENTS
`src/app`	116	11,116	Pages (`page.tsx`), layouts and the 81 API route handlers under `src/app/api`
`src/components`	118	35,299	React components, almost all under `components/quantifi` , plus the Shorts deck
`src/lib`	127	31,299	Domain logic: data clients, scoring, valuation, filings pipeline, news, AI, community, analytics
`src/data`	16	4,117	Curated reference data: ideas, playbooks, plans, heatmap universe, Indian ETFs and banks
`scripts`	70	14,881	58 test suites, ingestion scripts and fixtures
`.github/workflows`	1		Scheduled NSE insider ingestion
`public`			Logos, icons and a vendored DOMPurify build

The largest modules show where the complexity sits. In the library: `yahooFundamentals.ts` (1,978 lines, the scorecard), `balanceSheet.ts` (1,476, the sector checklists), `incomeFlow.ts` (1,394, the revenue Sankey) and `listingRank.ts` (1,201, choosing the right listing of a company). In the interface: the landing chart, the portfolio manager, the community composer and the stock explorer each exceed 900 lines.

5 Runtime, deployment and scheduled work

How a request is served

Every request first meets `src/middleware.ts`, which runs at Vercel's edge. It does two things. It generates a fresh random nonce and attaches a strict Content-Security-Policy that only trusts scripts carrying that nonce; and it decides whether the path is public. Public paths are listed in `src/lib/publicRoutes.ts`, which the sitemap also reads, so the gate and what is advertised to search engines cannot disagree. A gated path without a session cookie is redirected to sign-in.

Middleware only checks that a cookie exists. The signature is verified again inside every data route and server component, so a forged cookie passes the edge but receives no data. The page then renders on the server; its interactive sections are client components that fetch their own data from the API routes.

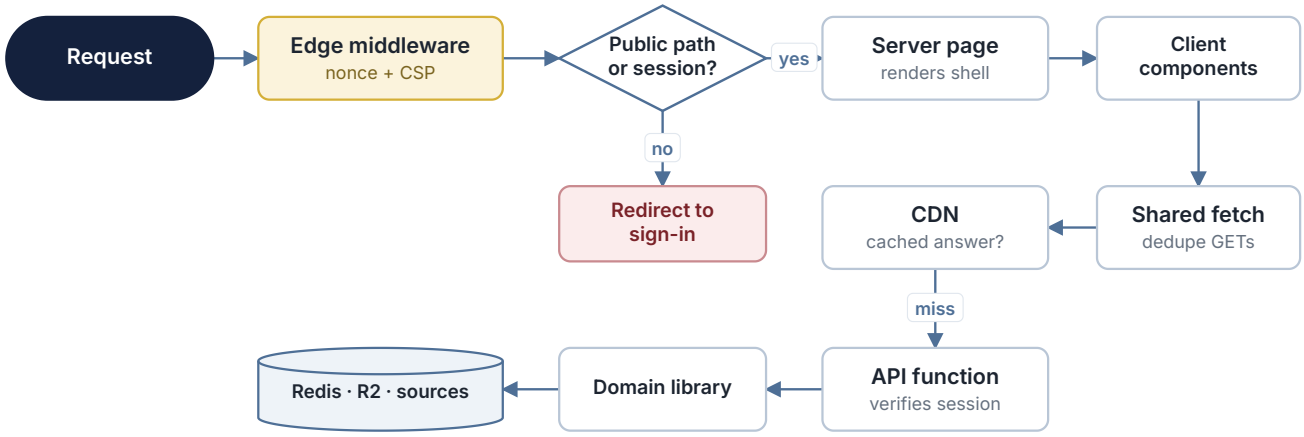


Figure 5.1 Request lifecycle. The edge decides access; the CDN absorbs repeat reads; only a miss reaches a function, which re-verifies the session before touching data.

Caching, three layers deep

LAYER	WHERE	WHAT IT PREVENTS
Shared fetch	`src/lib/sharedFetch.ts`, installed once in the root layout	Ten components on the stock page each fetching `/api/company/<symbol>`: identical in-flight GETs collapse into one request. Per-account routes are excluded so two readers can never share an answer.
CDN	`src/lib/httpCache.ts` sets `s-maxage` and `stale-while-revalidate`	Repeat requests re-invoking the function; the Undervalued Finds scanner fires about 35 score requests at once and only the first cold one per ticker runs.
Redis	Per-feature keys with TTLs	Re-fetching slow or rate-limited upstreams: Yahoo's crumb handshake, exchange master files, news summaries, executive profiles, insider snapshots.

Scheduled work

Data that is published on a calendar is pulled on a calendar. Vercel Cron calls six job routes, which require `CRON_SECRET` when it is set (see Chapter 21), and a GitHub Actions workflow runs the NSE insider ingestion from outside Vercel's network, writing directly to Redis.

JOB	SCHEDULE (UTC)	WHAT IT DOES
`/api/cron/insider-in`	Daily 13:30	Indian insider disclosures (SEBI PIT filings) from NSE
`/api/cron/insider-tw`	Daily 01:00	Taiwan insider holdings changes from TWSE and TPEx
`/api/cron/securities-master`	Sundays 17:00	Rebuilds the listed-universe master from both Indian exchanges
`/api/cron/filings-banks`	Daily 19:00	Polls Indian bank filings and refreshes balance-sheet facts
`/api/cron/filings-india`	Daily 20:00	Polls the rest of the Indian market's results filings, in cursor-sized batches
`/api/cron/filings-us-banks`	Daily 20:30	Official FR Y-9C balance sheets for US bank holding companies
`/api/cron/filings-rbi`	On demand	Imports the RBI's bank-wise statistical tables (route exists, not scheduled)
GitHub Actions: ***Ingest Indian insider disclosures***	Weekdays 14:00	Downloads NSE's market-wide insider CSV and writes per-symbol records to Redis; can be run manually with a look-back window and a reset flag

Configuration

All secrets and switches come from environment variables, set in Vercel and, for the ingestion workflow, in GitHub repository secrets. About fifty names are read across the code. They fall into a few families:

FAMILY	VARIABLES (NAMES ONLY)
Identity	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `AUTH_SECRET` (signs sessions and derives the storage encryption key)
Access	`PRO_EMAILS`, `ADMIN_EMAILS` (allowlists; empty means nobody)
Storage	`KV_REST_API_URL`, `KV_REST_API_TOKEN`, `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET`
Payments	`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_PLAN_PRO`, `RAZORPAY_WEBHOOK_SECRET`, `RAZORPAY_TRIAL_DAYS`
AI	`AI_QUANTIFI`, `AI_QUANTIFI_MODEL`, `AI_QUANTIFI_WEB_MODEL`, `AI_QUANTIFI_WEB_TOOL`
Data sources	`EDGAR_USER_AGENT`, `SCRAPER_API_KEY`, `LOGO_DEV_TOKEN`, `FMP_API_KEY`, `EODHD_API_KEY`, `BSE_API_KEY`, `NSE_FEED_TOKEN`
Jobs	`CRON_SECRET`, `FILINGS_INGEST_SECRET`, `INSIDER_WINDOW_DAYS`, `BSE_WINDOW_DAYS`

Two example files document the optional groups: `.env.ai.example` for the assistant and `.env.filings.example` for the filings pipeline. Several integrations are written to stay dormant until their variables exist, so the application builds and runs with only the identity and storage variables set.

III

PART III

Frontend

The interface a reader uses: the application shell and design system, then each surface in turn, and the client-side plumbing that holds them together.

6 Application shell and design system

7 Home, discovery and markets

8 Stock analysis

9 Portfolio and watchlist

10 News and Financial Shorts

11 Community

12 Client infrastructure and component inventory

6 Application shell and design system

Every page is wrapped by `src/app/layout.tsx`. The root layout renders a live market strip (*MarketPulse*) above the navigation bar, the page itself, and a footer. It also mounts four invisible pieces of infrastructure: the shared-fetch layer, a page-view tracker, the launch-offer popup, and Vercel Analytics. Structured data for search engines (organisation, website and software application) is emitted as JSON-LD, and the theme script runs with the request's CSP nonce so it can execute under the strict policy.

The shell, top to bottom

ELEMENT	COMPONENT	BEHAVIOUR
Market strip	`MarketPulse`	A ticker of world indices, volatility, bitcoin and gold with the day's change, a row of today's biggest movers, and a last-updated line. Rendered on the server for the first paint; its loading placeholder has identical heights so the page never jumps when quotes arrive.
Navigation	`Navbar`, `DiscoverNav`	Six entries plus Subscribe; a command search; the notification bell; the account menu. The logo switches from white to gold for Pro subscribers.
Search	`CommandSearch`	Opens as a full panel (keyboard shortcut Ctrl/Cmd K) rather than a dropdown, so recent searches, companies grouped by listing, funds and pages all fit. Results come from `/api/symbol-search`.
Alerts	`NotificationBell`	Watchlist price alerts and community activity (likes, replies, follows); every alert links to the in-app page for its ticker.
Offer	`LimitedOfferPopup`	A one-per-session launch offer, never shown to Pro subscribers or to anyone not signed in.
Footer	`Footer`	Company, legal and plan links.

Visual language

The palette is defined once in `tailwind.config.ts` as semantic tokens: a near-black *ink* scale for surfaces, *gold* (#D4AF37) for brand and emphasis, *teal* for interactive accents, *up* (#34D399) and *down* (#FB7185) for market direction, and a single violet, *verified* (#A78BFA), reserved for the community verification tick so that it cannot be confused with a highlighted figure.

Typography is split by role. **Inter** carries prose and headings, with the single-storey alternates that keep it legible at small sizes. **IBM Plex Mono** is bound to the `font-mono` utility used on every price, ratio and percentage, so a figure always reads as a quantity rather than as a word in a sentence. Figures are tabular so columns line up.

The site opens in dark mode and switches to light only for a reader who asks; the choice is remembered. A design system in `src/components/quantifi/System.tsx` provides the primitives every page uses (panels, section headings, eyebrows, stat tiles, tables), composed only from semantic tokens so a panel is identical in both themes.

THE RULE THE DESIGN SYSTEM ENFORCES

The file opens with an audit: 98 components had each invented their own panel, producing 291 uses of one translucent background, 272 of one border, 239 pill shapes and only eleven tables. Its rule is that *information lives on the page*. A container must earn its place by making something clearer than spacing and a rule could, and since most financial information is a label and a number, the default unit is a row, not a card.

Responsive behaviour

Layouts are mobile-first Tailwind. Wide content (financial statements, peer tables) was reworked over several iterations to use the available width rather than asking for a horizontal swipe; the landing chart and the Shorts deck size themselves from measured viewport space rather than fixed breakpoints, and both have dedicated phone layouts.

7 Home, discovery and markets

The landing page

Visitors who are not signed in see `LandingAscent`, the largest component in the codebase. It is a single ascending price chart that advances as the reader scrolls, carrying the product's four claims past them and ending at the sign-in control. The chart's series comes from `src/lib/marketTape.ts`, whose brief was realism: it generates a path with volatility clustering, realistic candle bodies and wicks, and no repeated bars, and 104 test assertions hold it to that. The page runs on phones and stops short of taking the whole screen.

The signed-in home

SECTION	COMPONENT	CONTENT
Market summary	`MarketSummary`	A short written account of what the world's markets did today, one market at a time, with the moment its figures belong to
Heatmap	`MarketHeatmap`	Treemap of the largest companies in nine markets (US, India, UK, Japan, Hong Kong, Germany, France, Canada, Australia); India by default. Tiles are sized by market capitalisation and coloured by the day's move
Portfolio command centre	`PortfolioCommandCenter`	Today's move, holdings and weights for the reader's book in one panel, from one set of quotes
Financial Shorts	`ShortsPromo`	An entry into the headline deck

The heatmap is Quantifi's own. It replaced an embedded third-party widget because clicking a company in the widget sent the reader to another website at exactly the moment they showed interest. Tile labels use the company's name rather than its code, and their size is computed from measured letter widths so a label never overflows its tile (`src/lib/heatmapLabel.ts`, 55 assertions).

Discover

SURFACE	WHAT IT DOES
Investing Ideas	Curated themes (`src/data/ideas.ts` , the largest data file) with thesis tests whose signals are labelled encouraging, neutral or warning. Free accounts open the first two featured ideas; the rest are Pro.
Playbooks	Step-by-step research playbooks, opened in a modal from an idea
Rare Finds	Curated names enriched live, for example with the most recent insider purchase read from SEC Form 4 filings
Undervalued Finds	A live scan of roughly 35 names through the scoring engine, with the analyst bear, base and bull targets shown against price
Screener	Liquid names scored live from current fundamentals, filterable by axis scores and by presets such as *quality compounders* (track record and balance sheet both strong)
Markets	The market as a whole: valuation, returns, the sectors that moved it and the companies that drove them, aggregated from live quotes and index history

Other tools

The tools area includes an XIRR calculator, which solves the money-weighted annual return of dated cash flows with a bisection-then-Newton solver, a currency converter, and sector browsing. *Famous Takes* collects well-known investor narratives. The insider page aggregates US Form 4 filings, Indian SEBI disclosures, Taiwanese holdings changes and US legislators' trades, with a quarterly buy-versus-sell chart.

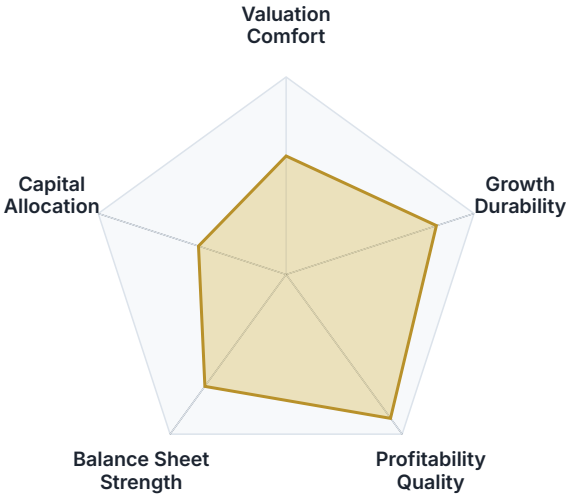
8 Stock analysis

The stock analysis page is the product's centre of gravity and is assembled by `StockExplorer` . It works for any listed company or fund the search can resolve, in any supported market, and decides at load time which sections a security can support: a fund gets a fund X-ray instead of financial statements, a bank gets a bank's statement layout, and a section with no data is omitted rather than shown empty. A sticky rail (`StockSectionNav`) lists only the sections that actually rendered.

SECTION	MAIN COMPONENTS	WHAT IT SHOWS
Chart	`PriceChart` , `StockHero`	Price history from one day to maximum, candles or line, corporate events marked on the chart; the window decides the bar interval
Snowflake score	`CompanySnapshot`	The five-axis score with one dot per check (pass, fail, or hollow when unmeasured) and the reason for each
Fund X-ray	`EtfSnapshot`	For funds: holdings, sector and country mix, cost and size
Overview and financials	`CompanyDetails`	Income statement, balance sheet and cash flow, annual or quarterly, with a template chosen by kind of company
Management	`ExecRow`	Executives matched to verified Wikidata profiles (education, career, portrait)
Financial health	`DebtEquityHistory`	Debt against equity over time, under the balance-sheet model that fits the company
Key statistics	`CompanyVitals`	Valuation multiples, returns, margins, size

SECTION	MAIN COMPONENTS	WHAT IT SHOWS
Valuation	`KeyValuationMetric`, `FairValueBars`, `ValuationSandbox`	The lens that suits the sector, price against modelled value, the value's history, and a sandbox to change the assumptions
Analyst view	`AnalystConsensus`	Consensus rating and target range
Future growth	`FutureGrowth`	Consensus revenue and earnings forecasts, with the horizon actually used stated beside each rate
Revenue and expenses	`RevenueBreakdown`	A Sankey flow from revenue to profit that conserves every unit
Ownership	`ShareholdingStats`	Insider, institutional and public ownership as a ring
Competitors and peers	`Competitors`, `PeerComparison`	Peer snowflakes and a head-to-head table with a rebased price chart
News	`CompanyNewsSection`	Stories matched to this company
Insider trades	`InsiderActivity`	Recent insider transactions and a quarterly buy/sell chart
Top shareholders	`TopShareholders`	Largest institutional and fund holders
My notes	`MyNotes`	Private notes on the company, kept in the reader's browser

The score



Valuation Comfort
Is the price reasonable for what the business earns and owns?
P/E, PEG, P/S, price over modelled fair value, free-cash-flow yield, analyst target gap

Growth Durability
Is the business growing, and is the growth likely to last?
Revenue and earnings growth, consensus forward growth

Profitability Quality
Does it earn good returns on the capital it uses?
Return on equity and assets, margins, profitable-year record

Balance Sheet Strength
Can it survive a bad year?
Sector-specific checklist: nine checks for industrials; capital, asset quality and liquidity for lenders

Capital Allocation
What does management do with the cash?
Dividends, buybacks against dilution, reinvestment

Figure 8.1 The five axes of the Quantifi Score. The shape shown is illustrative. Each axis is scored from a set of named checks, and an axis with too few measured checks is reported as insufficient rather than scored on what is left.

Each axis is a list of checks, each check a threshold on one measured figure with its definition attached. An axis carries a sufficiency rule: a minimum number of evaluated checks and, for the balance sheet, at least one check from each required domain (for an industrial company, leverage and liquidity). The score is stored on a six-point scale and presented on a ten-point feel.

Gates on the page

The page is public, so a visitor arriving from the landing search can read one whole analysis. After that the score is behind a sign-up modal, and a free account is limited to two full analyses a day, counted on the server per account and UTC day. A floating action dock (`StockActionDock`) holds the page's actions: add to a portfolio, add to the watchlist, open private notes, and open the contextual AI assistant.

THE CONTEXTUAL ASSISTANT

`StockAiAssistant` answers questions about the company on screen using a financial snapshot built from the page's own data, plus bounded external research. Questions that only make sense against the reader's holdings (*how much of my book is this?*) are answered by attaching the selected portfolio, which is built in the browser and sent with the question; the server keeps none of it.

9 Portfolio and watchlist

`PortfolioManager` (1,101 lines) manages any number of named portfolios. Holdings are entered by searching for the company rather than typing a symbol, recorded as lots (so buying the same stock twice is two lots, merged into one position for display), and can be edited or partly sold. Holdings can be exported as a spreadsheet-ready CSV.

Arithmetic that survives a mixed book

A portfolio holding Indian and US shares cannot be added up without exchange rates. The original code added rupees to dollars and reported a small-cap as 72% of a book where it was about 3%. `src/lib/portfolioFx.ts` now converts every row into one display currency before any weight, total or gain is computed, and a row whose rate has not arrived is shown as unavailable rather than converted at one-to-one. One function computes the weights for both halves of the page so the allocation ring and the holdings table can never disagree (`src/lib/portfolioBook.ts`).

PANEL	COMPONENT	WHAT IT SHOWS
Allocation	`AllocationDonut`	Five largest positions and an exact *Other*, with each slice's day move; hovering a slice explains it
Value over time	`PortfolioValueChart`	The book's value history on lightweight-charts
Benchmarks	`PortfolioAlpha`	Time-weighted return against the Nifty 50, S&P 500, FTSE 100, Nikkei 225 or Hang Seng over six months or a year, and what the difference was made of
Quality	`PortfolioSnowflake`	The book's weighted score snowflake
Diversification	`Diversification`	Concentration by holding and by sector
Assistant	`PortfolioAiAssistant`	An assistant that can read the selected portfolio, run the screener and search the web
Sharing	`PortfolioSharing`	Publish the book to the community profile as weights only, or take it down

Measuring against the market

The benchmark comparison is a time-weighted return: positions that enter part-way through the window are treated as deposits rather than gains, so adding a second holding does not show up as a 100% return. The same calculation powers published portfolios on community profiles. Because a return is unchanged when every share count is multiplied by a constant, a published book's benchmark comparison can be rebuilt from its weights and today's prices without ever learning the owner's amounts.

Watchlist

The watchlist shows each tracked name with its score snowflake, price, distance from modelled value, seven-day and three-month moves, a sparkline and the analyst target. It is searched by name with the same typeahead as the research page. Free accounts can track five names.

> WHERE THE DATA LIVES
>
> Portfolios, watchlist and alerts use one hook, `useSyncedState`. Signed out, the value lives in the browser. Signed in, it is mirrored to `/api/store`, encrypted at rest, and follows the account across devices. Each copy carries the time it last changed, so the newer copy wins when a device reconnects.

10 News and Financial Shorts

The news page reads live feeds and shows a lead story followed by a list, with source and headline set apart. Opening a story produces a written read (`NewsImpact`): what changed, why it matters, the causal path, the companies affected, what to watch, and what would undercut the read. The read is structured rather than a paragraph, and the *what would undercut it* line is always present.

Financial Shorts

Shorts are the day's market headlines as a vertical deck, one story to a screen (`src/components/quantifi/shorts`). The deck uses native scroll-snap rather than a carousel library, so momentum scrolling, trackpads, find-in-page and screen readers keep working. Each card has a visual chosen by `storyTemplate.ts` (a price chart for a company story, an abstract field for a market story), the headline in full (never truncated), a summary of up to about sixty words, real likes and a comment thread. Liking turns the heart into the Quantifi mark.

DECISION	RULE IN CODE
Which market a story is about	Decided from the wire the story came from before any name matching (`newsMarket.ts`)
Which company it names	A one-word company name must be written as a name; *Target* in *price target* is not Target Corporation
What a summary may say	Only numbers that appear in the source; a summary that adds a figure is rejected
When the summary is unavailable	The card shows the headline rather than waiting on a model that is not answering
How the card is sized	From measured parts, so the summary never bleeds past its box

11 Community

The community is a feed of investment theses. Its premise, stated in the composer's code, is that the order on screen is the order of the argument: what the author thinks comes first, and the evidence is attached under it.

Writing a post

`CommunityComposer` starts as a single line and expands when there is something to say. A post has a required title, a body in a light markdown dialect (headings, lists, quotes, emphasis), up to eight tagged companies and up to six attachments. Attachments are *figures lifted from the app*, frozen at the moment of posting with their source and date so a thesis cannot silently re-render at different numbers:

ATTACHMENT	CONTENT
Portfolio	The author's allocation, drawn as a split
Chart	A price chart, as candles
Image	An uploaded picture (PNG, JPEG, GIF or WebP, up to 5 MB)
Figures, Score	Key statistics or the five-axis score
Income, Balance, Cash flow	Statement extracts, laid out by kind of company
Holders, Insider	Ownership ring and insider activity
Fund	A fund's X-ray
Tag, Link	A tagged company, or up to four links to what someone said elsewhere

Reading, following and identity

The feed (`CommunityFeed`) puts posts by people the reader follows first, then the rest by recency. Profiles (`/community/u/[handle]`) show a picture, a self-written role and bio, a link, posts, follower counts and, if the owner chose to publish it, their portfolio drawn exactly as the owner sees it with benchmark comparison. Pictures and names are both links to the profile. Two accounts carry a violet verification tick. Likes, replies and follows generate notifications.

> RULES, NOT JUST SCREENS
>
> The community's decisions live in importless modules with their own tests: what a post may contain (`communityPost.ts` , 71 assertions), what a profile may say and which handles are reserved (`communityProfile.ts` , 137), how following orders the feed (`follow.ts` , 45), what counts as news for the inbox (`communityInbox.ts` , 37) and what an upload must be (`communityImage.ts` , 29).

12 Client infrastructure and component inventory

Plumbing shared by every page

PIECE	FILE	JOB
Shared fetch	`src/lib/sharedFetch.ts`	Wraps `window.fetch` once; identical in-flight GETs to market-data routes collapse into one request and the answer is reused briefly
Synced state	`src/lib/useSyncedState.ts`	Browser storage when signed out, account storage when signed in, newest-wins reconciliation
Pro status	`src/lib/useProStatus.ts`	Reads the session once and exposes plan status for gates and the logo

PIECE	FILE	JOB
Watchlist and portfolios	`useWatchlist`, `usePortfolios`, `usePortfolioBook`	Typed access to the synced collections and their derived book
Usage beacon	`src/lib/track.ts`	Fire-and-forget events for the five funnel steps; never blocks a render or throws
Formatting	`src/lib/format.ts`	Numbers, currencies and Indian lakh/crore grouping in one place (68 assertions)

The largest components

COMPONENT	LINES	ROLE
`LandingAscent`	1,119	Scroll-driven landing chart
`PortfolioManager`	1,101	Portfolio page
`CommunityComposer`	920	Post editor with attachments
`StockExplorer`	912	Stock analysis page assembly
`PostEvidence`	854	Renders every attachment type, shared by composer and feed
`CommandSearch`	826	Full-panel search
`CompanySnapshot`	804	Score and checks
`Markets`	776	Markets overview
`InsiderActivity`	775	Insider tables and charts
`Cards`	761	Shared card primitives and ticker links
`TradingIdeas`	760	Investing Ideas
`AdminOps`	670	Operations dashboard
`CommunityProfile`	616	Profile page
`CompanyDetails`	609	Financial statements
`PriceChart`	603	Interactive price chart
`FutureGrowth`	580	Forecast section
`NewsFeed`	564	News page
`DebtEquityHistory`	549	Leverage over time
`Screener`	531	Screener
`MarketHeatmap`	521	Treemap heatmap

In total there are 118 component files and about 35,300 lines of interface code. Charts that encode a claim (allocation rings, Sankeys, heatmap tiles, sparklines, the snowflake) are drawn in hand-written SVG so the drawing rules can be tested as data; interactive price series use lightweight-charts.

IV

PART IV

Backend

The server half of the application: every API route, where the data comes from, where it is kept, the analytical engine, the filings pipeline, AI, payments, and the security model.

13 API surface

14 Data acquisition and caching

15 Storage and data model

16 The analytical engine

17 The filings pipeline

18 Insider, news and AI services

19 Identity, access and payments

20 Community, analytics and administration

21 Security model

13 API surface

The backend is 81 Next.js route handlers under `src/app/api`. They are thin by design: a route authenticates, validates its input, calls the domain library and shapes the response, and the logic that decides anything lives in `src/lib` where it can be tested. Read-only market routes set CDN cache headers; per-account routes are marked private and uncached.

81	51	7	30
route handlers	read-only (GET)	scheduled job routes	accept writes

MARKET DATA AND COMPANY RESEARCH

ROUTE	METHOD	PURPOSE
`/quote/[symbol]`	GET	Live price and day change; cached at the CDN for one minute
`/timeseries/[symbol]`	GET	Daily price history from Yahoo's chart endpoint, falling back to Stooq; real data only
`/company/[symbol]`	GET	Company profile and financials; cached ten minutes
`/score/[symbol]`	GET	The five-axis Quantifi Score with every check and its reason; records fair-value history
`/statements/[symbol]`	GET	The three statements for annual or quarterly periods
`/fundamentals/[ticker]`	GET	Free fundamentals from SEC EDGAR for US filers; cached one hour
`/income-flow/[symbol]`	GET	Revenue-and-expenses flow under the reporting model for the industry
`/future-growth/[symbol]`	GET	Analyst forecasts plus the peer medians the section compares against
`/fair-value-history/[symbol]`	GET	Recorded cash-flow value against price, plus a labelled reconstruction of earlier years
`/pe-history/[symbol]`	GET	Approximate historical P/E, to judge a stock against its own history
`/peers/[symbol]`	GET	Competitors: curated overrides and filtered candidates
`/peer-cards/[symbol]`	GET	Descriptions, sectors and moves for the peer cards
`/etf/[symbol]`	GET	Fund analysis; answers `available: false` for an ordinary stock
`/events/[symbol]`	GET	Dated corporate events for the chart: SEC 8-K items, dividends and splits
`/inst-ownership/[symbol]`	GET	13F institutional ownership; cached one hour
`/exec-profile`	GET	An executive's education, career and portrait from Wikidata and Wikipedia
`/logo/[symbol]`	GET	Company logos relayed through Quantifi's own origin
`/portrait`	GET	Executive portraits from Wikimedia Commons, relayed through Quantifi's own origin
`/symbol-search`	GET	Typeahead search across providers, with one normalised label per security type
`/resolve`	GET	Name to ticker: a curated map first, then Yahoo search
`/fx`	GET	Exchange rates from the ECB's Frankfurter service, falling back to Yahoo

MARKETS AND THE FRONT PAGE

ROUTE	METHOD	PURPOSE
`/markets`	GET	Whole-market view for one region, aggregated from company quotes
`/markets/returns`	GET	Sector returns over a chosen window
`/heatmap`	GET	Heatmap tiles for a region
`/market-summary`	GET	What each market did today, written every six hours and shared by every reader
`/pulse`	GET	The market strip's live data
`/notifications`	GET	Intraday price-move alerts for the bell

NEWS AND SHORTS

ROUTE	METHOD	PURPOSE
`/news-for`	GET	Resolves a company's display name so a focused news query can be built
`/news-shorts`	GET	The Shorts deck, with every card's summary written before it answers
`/news-summary`	POST	One card's summary on demand
`/news-read`	POST	The structured read behind a story, written when it is opened
`/news-reactions`	GET POST	Likes on a news card, as set membership
`/news-comments`	GET POST	Threaded comments on a story

INSIDER AND OWNERSHIP

ROUTE	METHOD	PURPOSE
`/insider/[ticker]`	GET	Insider transactions for one company (US Form 4, India, Taiwan)
`/insider-feed`	GET	Market-wide insider feed
`/insider/status`	GET	Diagnoses why insider data is missing, across four independent failure points
`/insider/probe`	GET	Tests which Indian data sources answer this deployment
`/congress-trades`	GET	US legislators' disclosed trades

FILINGS PIPELINE

ROUTE	METHOD	PURPOSE
`/filings/[symbol]`	GET	Facts held from a company's own filings, with rejected facts and their reasons
`/filings/health`	GET	One URL that says where the filings chain is broken
`/filing`	GET	Fetches a filing document through a strict host allowlist (SSRF guard)
`/filings/discover`	POST	Find and ingest a company's filings from the admin page, diagnostic first
`/filings/ingest`	POST	Token-authorised ingestion with constant-time token comparison

AI

ROUTE	METHOD	PURPOSE
`/ai/stock`	POST	The contextual stock assistant: financial snapshot, bounded web research, model call
`/ai/portfolio`	POST	The portfolio assistant, able to read the selected book, run the screener and search the web

ACCOUNTS, PLANS AND PAYMENTS

ROUTE	METHOD	PURPOSE
`/auth/login`	GET	Starts Google sign-in; detects in-app browsers that Google blocks
`/auth/callback/google`	GET	Completes sign-in, sets the signed session, distinguishes sign-up from return
`/auth/session`	GET	Session hint for rendering; never a permission
`/auth/logout`	GET	Clears the session
`/store/[key]`	GET PUT	Encrypted per-account storage for portfolios, watchlist and alerts only
`/free-analyses`	GET POST	The free plan's daily analysis quota, per account
`/create-order`	POST	Razorpay order, step one of one-time checkout
`/verify-payment`	POST	Verifies a one-time payment's signature, then grants Pro
`/checkout`	POST	Starts a recurring Pro subscription
`/razorpay/verify`	POST	Verifies a subscription checkout, then grants Pro
`/webhook`	POST	Razorpay lifecycle events keep Pro access in sync
`/claim-pro`	POST	Launch-offer grant: a week of Pro with no card

COMMUNITY

ROUTE	METHOD	PURPOSE
`/community/posts`	GET POST	The feed (followed authors first) and new posts
`/community/posts/[id]`	GET DELETE	One post; deletion by its author or the operator
`/community/comments`	GET POST	Replies under a post
`/community/reactions`	POST	Likes, as set membership
`/community/saved`	GET POST	Posts a reader kept
`/community/follow`	GET POST	Who an account follows
`/community/profile`	GET PUT	Your own profile and handle
`/community/profile/[handle]`	GET	Somebody's public profile, posts and published portfolio
`/community/portfolio`	GET POST	Publish or withdraw your portfolio (weights only)

ROUTE	METHOD	PURPOSE
`/community/notifications`	GET POST	Who liked, replied to or followed you
`/community/attachment`	POST	Turns a figure (for example a P/E) into a frozen card while composing
`/community/image`	POST	Image upload: byte sniffing, size cap, content-hashed name
`/community/image/[name]`	GET	Serves an uploaded image by its hash
`/community/ask`	GET POST	Questions to the team; the inbox is owner-only

OPERATIONS

ROUTE	METHOD	PURPOSE
`/track`	POST	Page-view and funnel beacon; identity taken from the signed cookie
`/admin/ops`	GET	Deployment health for staff: funnel, conversions, pipelines
`/admin/run/[job]`	POST	Runs a scheduled job on demand from the admin page
`/cron/*`	GET	Seven protected job routes (Chapter 5)

14 Data acquisition and caching

Quantifi holds no market data of its own. Prices, fundamentals, filings and news are fetched from the organisations that publish them, normalised, cached and computed on. The table lists every source the server contacts, taken from the code rather than from documentation.

SOURCE	USED FOR	ACCESS
Yahoo Finance	Quotes, price history, fundamentals, statements (through its timeseries service), analyst estimates, ownership, search	Unofficial endpoints with a cookie-and-crumb handshake
NSE and BSE	Indian corporate announcements, results filings (XBRL), insider disclosures, listed-universe masters	Public web endpoints and downloads; a proxy service for blocked hosts
SEC EDGAR	US filings, company facts, Form 4 insider transactions, 8-K events, the US company universe	Public API with a declared contact in the user agent
TWSE and TPEx	Taiwanese insider holdings changes	Open-data APIs
RBI and FFIEC	Indian bank-wise statistical tables; US bank holding company FR Y-9C balance sheets	Published tables and bulk files
Google News RSS	Headlines for the feed and Shorts	Public RSS
Stooq	Fallback daily price history	Public CSV
Frankfurter (ECB)	Exchange rates	Keyless public API
Wikidata, Wikipedia, Wikimedia	Executive biographies and portraits	Public APIs
Congressional trade datasets	US legislators' disclosures	Public datasets

SOURCE	USED FOR	ACCESS
DeepSeek	Summaries, news reads, assistants, web search	Commercial API
Financial Modeling Prep	Institutional ownership and legislators' trades	Dormant unless a key is configured

Living with an unofficial upstream

Yahoo's statement endpoint now requires a cookie and a *crumb* token, and it rate-limits the handshake that issues them. Originally every library module ran its own handshake, so one page load cost three and routinely tripped the limit, which users saw as *analysis not available* for perfectly valid companies. `src/lib/yahooCrumb.ts` centralises it: one crumb reused for about ten minutes per warm instance, several cookie hosts tried in turn, retries with backoff, and transparent refresh when Yahoo answers 401, 403 or 429.

A second lesson is recorded in the scorecard: Yahoo's `quoteSummary` statement modules now answer empty for most symbols, and the timeseries service is what actually serves statement lines. Derived measures (return on equity and assets, price to sales, PEG) are therefore computed from timeseries statements when the ready-made ratio is missing, with the definition of the derived figure carried alongside it.

Choosing the right listing

A company can trade in several places: HDFC Bank is HDFCBANK on the NSE, 500180 on the BSE and HDB in New York. `src/lib/listingRank.ts` decides which listing to show and how to group the rest under it. Its rule, reached after four recorded bugs (Nike resolving to Stuttgart, then HDFC Bank resolving to its New York receipt, then Take-Two resolving to Austria), is *the issuer's home-market primary ordinary share*. It is the third-largest module in the library and has 246 test assertions.

Caching policy by route

DATA	CDN FRESHNESS	REASON
Quotes	About 1 minute	Fresh enough for a day change; absorbs watchlist and portfolio re-renders
Score	60 seconds	Absorbs the Undervalued Finds fan-out while staying close to the live price
Company profile	10 minutes	Profiles and financials change slowly
SEC fundamentals, 13F ownership	1 hour	Filed quarterly
Market summary	6 hours (stored)	Written once and read by everybody
Per-account routes	Never	Marked private; excluded from the shared-fetch layer too

15 Storage and data model

There are two stores. **Upstash Redis** holds everything that changes: accounts' plan status, synced user data, community content, caches and analytics. **Cloudflare R2** holds what is written once and kept: original filing documents and uploaded images. Both are reached over HTTPS with hand-written clients; there is no ORM and no connection pool.

The Redis keyspace

PREFIX	HOLDS	NOTES
`portfolios:<email>`, `watchlist:`, `alerts:`	Synced user collections	Encrypted with AES-256-GCM; key built from the session, never the request
`pro:<email>`	Plan status	Active flag, subscription id, period end; launch grants carry an expiry
`free:<email>:<day>`	Free-plan analysis quota	Per account per UTC day
`community:post:`, `community:posts`	Posts and the feed order	Each post its own key; a list holds the order
`community:comments:`, `:likes:`, `:saved:`	Replies, likes, saved posts	Likes are set membership, so a double click cannot count twice
`community:following:`, `:followers:`	Follow graph	Two sets written together
`community:profile:`, `:handle:`	Profiles and username ownership	A handle is claimed with an atomic set-if-absent
`community:portfolio:<id>`	Published portfolios	Weights only; deleted on withdrawal
`community:inbox:`	Notifications	Likes, replies and follows, rate-limited per actor
`filings:*`	Filings index, facts, cursors, health, errors	Shaped like relational tables so a move to Postgres is a driver change
`insider:us:`, `insider:in:`, `insider:tw:`	Insider records per symbol and market	Written by jobs, read by the insider pages
`news:summary:`, `news:read:`, `news:likes:`	News summaries, reads, reactions	Summaries keyed by content hash
`stats:<day>:*`	Analytics	HyperLogLog visitor counts, aggregate paths and tickers; 90-day expiry
`quantifi:fvhist:`	Fair-value history	One point per company per day
`exec:`, `logo:`, `isin:`, `master:`	Reference caches	Executive profiles, logo domains, ISIN maps, exchange masters

Encryption at rest

`src/lib/secretBox.ts` seals portfolios, watchlists and alerts with AES-256-GCM using a key derived with scrypt from the session secret, which lives in the environment and never in the database. A leaked database token or a stray backup therefore yields ciphertext. The module documents its own limits: it is not end-to-end encryption, it cannot protect a compromised server, and it does not encrypt the signed-out copy in the browser, because a key the browser could use would have to sit beside the data. Records written before encryption existed are read as plain JSON and resealed on their next write.

The document archive

Filings are archived in R2 under a prefix of their own, keyed by content hash, with metadata stored alongside. R2 suits an archive that is written once and checked occasionally because it charges nothing for egress. Requests are signed with AWS Signature Version 4, implemented in about three hundred lines rather than importing an SDK of several megabytes into every cold start. Community images live under a separate prefix so that a lifecycle rule for one can never touch the other.

16 The analytical engine

The analytical engine is the part of the library that turns raw figures into judgements. Its modules share a structure: an *importless* file of pure functions (numbers in, numbers out) that a test script can compile directly, wrapped by a thin data-fetching layer. Each answers one question.

MODULE	QUESTION IT ANSWERS	TESTS
`yahooFundamentals.ts`	What is this company's five-axis score, and why?	via attachment and e2e suites
`valuationModel.ts`	Which valuation model fits this company, and what does it say?	85
`financialHealth.ts`	Which balance-sheet model should this company be read under?	155
`balanceSheet.ts`	Does it pass that model's checklist?	332
`incomeFlow.ts`	How does revenue flow to profit, drawn honestly?	338
`futureGrowth.ts`	What do analysts forecast, over what horizon really?	95
`peerSelection.ts`	Is this candidate a genuine competitor?	regression suite
`statementTemplate.ts`	Which statement lines does this kind of company have?	37
`portfolioAlpha.ts`, `portfolioFx.ts`	Did the book beat the market, in one currency?	78, 41

The valuation router

A discounted-cash-flow model asks what cash a business will throw off. For a bank the question is malformed: lending is an operating outflow and deposits an operating inflow, so a bank's free cash flow measures where the accountants drew a line, not anything an owner could take out. The module records the consequence that prompted it: the old model printed a value of ₹3,171.55 for HDFC Bank beside a price-to-book reading of ₹472.58. Financial institutions therefore never enter the cash-flow model; they are valued on excess returns over the cost of equity (a residual-income model).

For everyone else a sector cluster chooses the headline multiple:

SECTOR CLUSTER	PRIMARY LENS
Banks and financial institutions	Price to book, with residual income
Broad market, Technology	Price to earnings
High-growth technology and SaaS, pre-profit	Enterprise value to sales

SECTOR CLUSTER	PRIMARY LENS
Telecom and infrastructure	EV to EBITDA (normalises very different debt loads and depreciation)
Commodities and resources	Asset value (book proxy)
Real estate	Net asset value (book proxy)

Balance-sheet models

Every company used to be judged against an industrial company's debt thresholds, so Power Finance Corporation, whose business is borrowing and lending on, was labelled *stretched* at 7.56 times gearing. `financialHealth.ts` now selects one of six books: **industrial**, **bank**, **US bank**, **NBFC**, **life insurer** and **general insurer**. Each has its own checklist in `balanceSheet.ts`. The industrial checklist has nine checks across leverage, liquidity and cash: net debt to EBITDA, interest burden, the short-term share of debt, equity to assets, cash conversion and capital-expenditure cover among them. A lender is checked on capital adequacy, asset quality (net NPA and provision coverage) and funding. Provision coverage is only compared like with like: the definition is stored with the value.

Honest forecasting and honest pictures

`futureGrowth.ts` starts from the fact that consensus data supplies two annual estimates and one long-term rate, which is less than it appears. A three-year rate cannot be read off two points, so where the long-term rate exists it is blended in for the unestimated years and the horizon actually used travels with the number; where it does not, the horizon is one year and the label says so. `incomeFlow.ts` draws the revenue Sankey under rules of conservation: every node's outflows sum to its value, a shortfall is drawn as an explicit *Other*, a derived cost is marked as derived, and a loss is drawn as a loss rather than as an empty space. It supports eight reporting models: industrial, bank, lender, insurance, fee-based, operating, cash-burn and a generic fallback.

17 The filings pipeline

The filings pipeline reads companies' own regulatory filings so that the most important figures come from the company's statement rather than from a data vendor's derivation. It lives in `src/lib/filings` and is the most carefully guarded part of the codebase, because its failures are silent: a parser that reads a nine-month column as a quarter, or lakhs as rupees, returns a plausible number that nothing downstream would question.

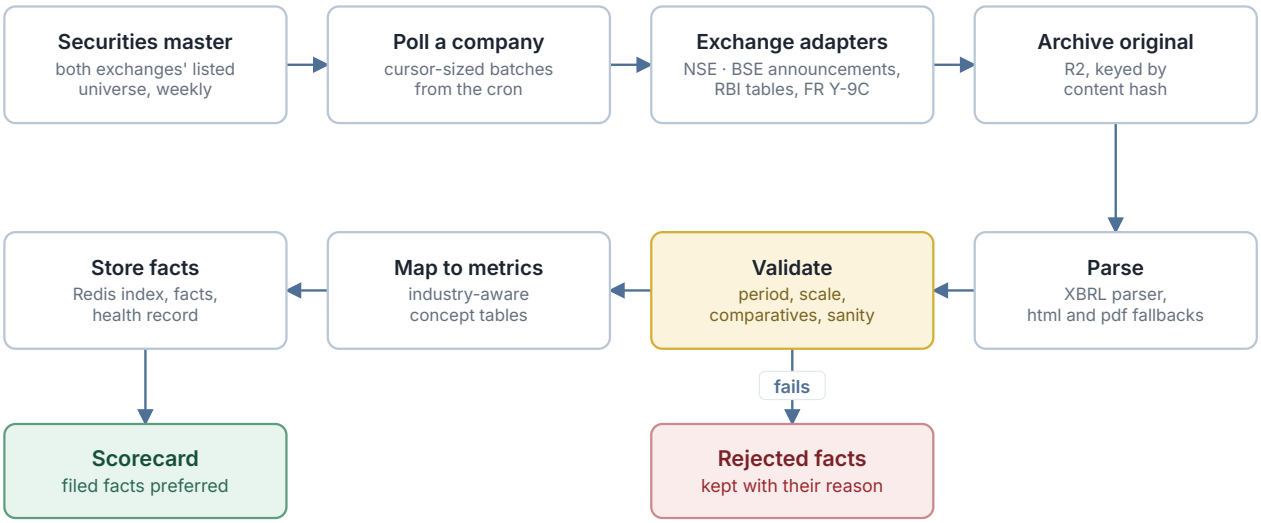


Figure 17.1 Filings pipeline. From the listed universe to facts the scorecard prefers over the data feed. A fact that fails validation is stored as rejected, with its reason, and is never silently converted to zero.

Ranking where a figure came from

RANK	EXTRACTION METHOD	WHY IT RANKS THERE
0	XBRL	The company's own tagged statement
1	Regulator table	The RBI's bank-wise tables: annual and standalone, but the reference the filing is checked against for asset quality and capital
2	HTML table	Structured, but not tagged
3	PDF text	Positional and fragile
4	OCR	Recognised characters
5	Language model	Last resort; never sent every filing

When two methods supply the same fact, the lower rank wins. Concepts are mapped per industry: an *InterestIncome* tag is a bank's top line and a manufacturer's treasury income, and a tag that only resolves in another industry's table is reported as a mismatch rather than accepted. Companies are keyed by ISIN or CIN, never by ticker, so a company's two exchange listings cannot become two issuers.

Operating the pipeline

Nightly jobs poll banks and then the wider market in batches that fit a serverless time limit, with a cursor in Redis to resume. When the parser improves, a self-heal pass re-parses archived documents so held facts pick up the fix, and stricter refusals are carried back over facts already stored. `/api/filings/health` reports where the chain is broken, and the admin page can discover and ingest a single company on demand. A test suite runs the whole path end to end: a filing goes in and a Balance Sheet Strength score comes out (142 assertions).

18 Insider, news and AI services

Insider data in four markets

MARKET	SOURCE	HOW IT ARRIVES
United States	SEC Form 4 filings	Read on demand per company, parsed with cheerio, cached in Redis
India	SEBI insider-trading (PIT) disclosures from NSE and BSE	A daily Vercel job and a weekday GitHub Actions run that downloads NSE's market-wide CSV; a proxy service as a fallback for blocked hosts
Taiwan	Six TWSE and TPEx holdings-change datasets	A daily job downloads, parses and stores per symbol
US legislators	Public disclosure datasets	Read on demand

`insiderVolume.ts` buckets transactions into quarters for the buy-versus-sell chart, and decides questions the chart silently claims: whether one person filing five times is one insider or five, and where a malformed or future date lands (52 assertions). `/api/insider/status` exists to answer *why is there no insider data* definitively, because the Indian pipeline has four independent points that can each stop silently.

News

News is matched to companies by `tickerDetect.ts`, which checks text against the full SEC universe of about ten thousand US companies, the complete NSE equity master, a supplemental list and index aliases. It runs on the server so the universe never ships to the browser, and it carries a growing list of traps: *Strategy* (formerly MicroStrategy) must not match *growth strategy*, *Target* must not match *price target*, and *fresh* must not match *fresh record high*. `newsMarket.ts` decides which market a story is about from the wire it came off before any name matching.

The model and its guardrails

Quantifi uses DeepSeek for four jobs: Shorts summaries, the structured read behind a story, the daily market summary, and two assistants. The web-searching assistants use DeepSeek's Anthropic-compatible Messages endpoint, which differs from its chat endpoint in path, headers, tool format and stream events; `deepseekWebSearch.ts` exists because a request in the wrong dialect returns a bare 404 that looks like an outage.

GUARDRAIL	WHERE	EFFECT
Numbers must come from the source	`newsShorts.ts`	A summary containing a figure absent from the headline or blurb is rejected
No article, no article summary	`newsShorts.ts`	RSS supplies headlines, not articles, so a card never pretends to summarise text it does not have
Every read has a counter-case	`newsImpact.ts`	The *what would undercut this* field is always present
Intent routing	`/api/ai/*`, tested by `test-ai-intent`	Questions about the reader's own holdings use the attached portfolio; questions about the world use bounded web research

GUARDRAIL	WHERE	EFFECT
Grounded snapshots	`portfolioAi.ts`, `portfolioSnapshot.ts`	The assistant is given the same holdings arithmetic the page shows, clamped and capped at 40 positions
Caching by content	`newsSummary.ts`	Summaries are keyed by a hash of their input, so one story is summarised once for everybody

19 Identity, access and payments

Sign-in

Authentication is Google OAuth 2.0 written directly against Google's endpoints. The login route sets a short-lived state cookie, redirects to Google, and the callback verifies the state, exchanges the code, reads the profile and issues a session cookie signed with HMAC. The cookie is HTTP-only, secure, same-site lax and lasts seven days. The callback also records whether this is the account's first sign-in, which is the only point at which a sign-up can be told apart from a return. The login route detects in-app browsers (LinkedIn, Instagram and others) that Google refuses, and asks the reader to open the page in a real browser rather than letting Google show an error.

Who may do what

CHECK	SOURCE OF TRUTH	NOTES
Signed in	Signed session cookie	Verified in every data route, not only at the edge
Pro	`PRO_EMAILS` allowlist, or `pro:<email>` in Redis	Written only by verified payments, the webhook or the launch grant
Staff	`ADMIN_EMAILS` allowlist	A separate list from Pro on purpose: a plan is not trust
Free quota	`free:<email>:<day>`	Server-side, so it survives refreshes and device changes

Allowlists fail closed: an empty or missing variable means nobody, never everybody, and there is no wildcard. That property has its own test suite because the dangerous failure is a permissive one.

Payments

Razorpay supports two paths, both written against its REST API with Node's crypto for signatures. A one-time purchase creates an order, opens Checkout in the browser, and verifies the returned signature on the server before granting a month of Pro. A subscription creates a recurring plan (with an optional trial that defers the first charge), verifies the checkout, and relies on the webhook to keep access in step with renewals, failures and cancellations. Every signature is compared in constant time. The client's word that a payment succeeded is never trusted, and a conversion is counted exactly once whichever path recorded it.

> LEFTOVER INTEGRATION
>
> A Stripe client (`src/lib/stripe.ts`) and the `stripe` package remain from an earlier payment design. Nothing imports the client; the dependency can be removed.

20 Community, analytics and administration

Community services

The community's routes hold the reads and writes; its decisions live in importless modules. Posts are validated against hard limits (120-character title, 5,000-character body, 8 tickers, 6 attachments, 4 links) and a 45-second cooldown per author; replies are capped at 1,200 characters with a 15-second cooldown and 500 per post. Uploads are identified by their first bytes, never by the declared content type; SVG is refused because it can carry script; files are capped at 5 MB and stored under the SHA-256 of their bytes. Handles that could impersonate staff (admin, moderator, official, team and others) are reserved, and a verified handle is never released for reuse.

Community identity is a one-way hash of the account's email truncated to 24 characters. There is no map back from a handle to an address, so no community feature can be used to name somebody's private storage. Publishing a portfolio writes a separate record derived on the owner's device, and a reader's request never touches the encrypted original.

First-party analytics

`src/lib/analytics.ts` records how people move through the site with no third-party tracker. Distinct visitors are counted with HyperLogLog, which answers *how many* to within about one percent while keeping no record of who. Paths and tickers are counted in aggregate. Signed-in activity is a per-day set of accounts that expires after 90 days. Five funnel steps are counted as distinct visitors per step:

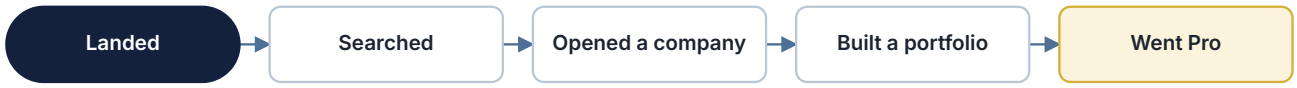


Figure 20.1 The conversion funnel. Steps are counted independently, so the drop between two steps is measured in people rather than page views.

Administration

`/admin` shows staff the state of the deployment rather than of any company: the funnel, Pro conversions with their source, pipeline cursors, filings health and insider freshness, and buttons to run any scheduled job on demand (the button calls the job server-side so the cron secret never reaches the browser). Every admin route re-derives staff status from the signed cookie and answers 404, not 403, to anyone else, so the response does not even confirm the route exists.

21 Security model

CONTROL	IMPLEMENTATION
Content Security Policy	Generated per request in middleware with a fresh nonce and `strict-dynamic`; only scripts carrying the nonce run. A report-only policy runs alongside for observation
Framing	`frame-ancestors 'self'` plus `X-Frame-Options: SAMEORIGIN`
Transport	HSTS for two years, including subdomains, preload-eligible

CONTROL	IMPLEMENTATION
Other headers	`X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, and a cross-origin opener policy that keeps payment pop-ups working
Sessions	HMAC-signed, HTTP-only, secure, same-site lax, seven days; verified in every data route
Per-account data	Keys built from the session's email, never from the request; encrypted at rest
Admin	Server-side re-verification, 404 to outsiders, fail-closed allowlists
User input	Profile links reject script and data schemes and credentials in the host, re-checked on read, rendered `nofollow`; posts are a typed block tree, never HTML
Uploads	Byte sniffing, SVG refused, size cap, content-hash names that cannot traverse paths
Server-side fetches	The filing fetcher takes a URL from the client and is guarded by a strict host allowlist against request forgery
Secrets in transit	Third-party images (logos, portraits) are relayed through Quantifi's origin so the CSP can stay narrow
Jobs and tokens	Cron routes check `CRON_SECRET` as a bearer header or a `?key=` parameter, but only when it is set; the filings ingestion token is compared in constant time

OPEN ITEMS

- The pinned Next.js 14.2.15 carries a published security advisory; npm flags it on install. Move to the patched 14.2 release.
- Two configuration fallbacks read `NEXT_PUBLIC_`-prefixed names (`NEXT_PUBLIC_FMP_API_KEY`, `NEXT_PUBLIC_PRO_EMAILS`). A variable with that prefix is compiled into the browser bundle, so populating either would publish its value. Both fallbacks should be deleted.
- All seven job routes accept any caller when `CRON_SECRET` is unset (`if (!secret) return true`), accept the secret in the query string, where it can reach request logs, and compare it with ordinary equality. If the variable is set in production the routes are protected; either way they should fail closed and read the header only.
- No external penetration test has been carried out.

V

PART V

Flowcharts

The system's most important paths drawn end to end: what a visitor goes through, how a score is computed, how a valuation is chosen, and how sign-in, payment, news and community posts work.

22.1 From visitor to subscriber

22.2 Computing a score

22.3 Choosing a valuation

22.4 Signing in

22.5 Paying for Pro

22.6 A headline becomes a card

22.7 Publishing a post

22.8 A nightly data job

22 Flowcharts

Each flowchart follows one path through the code. Rounded dark boxes are entry points, diamonds are decisions, gold boxes are the step the flow exists to protect, green and red boxes are outcomes, and cylinders are stores. Labels on arrows are the condition under which that branch is taken.

22.1 From visitor to subscriber

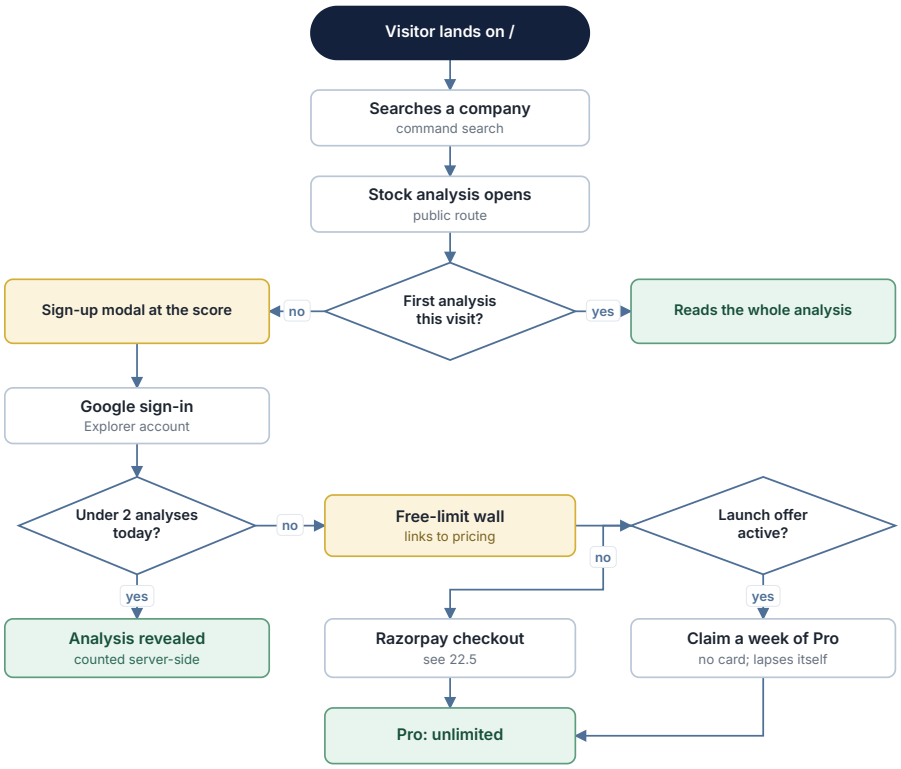


Figure 22.1 The acquisition path enforced by `publicRoutes.ts`, the stock page's reveal gate, `freeLimit.ts` and the plan flags in `plans.ts`. Each step is also a funnel event (Figure 20.1).

22.2 Computing a score

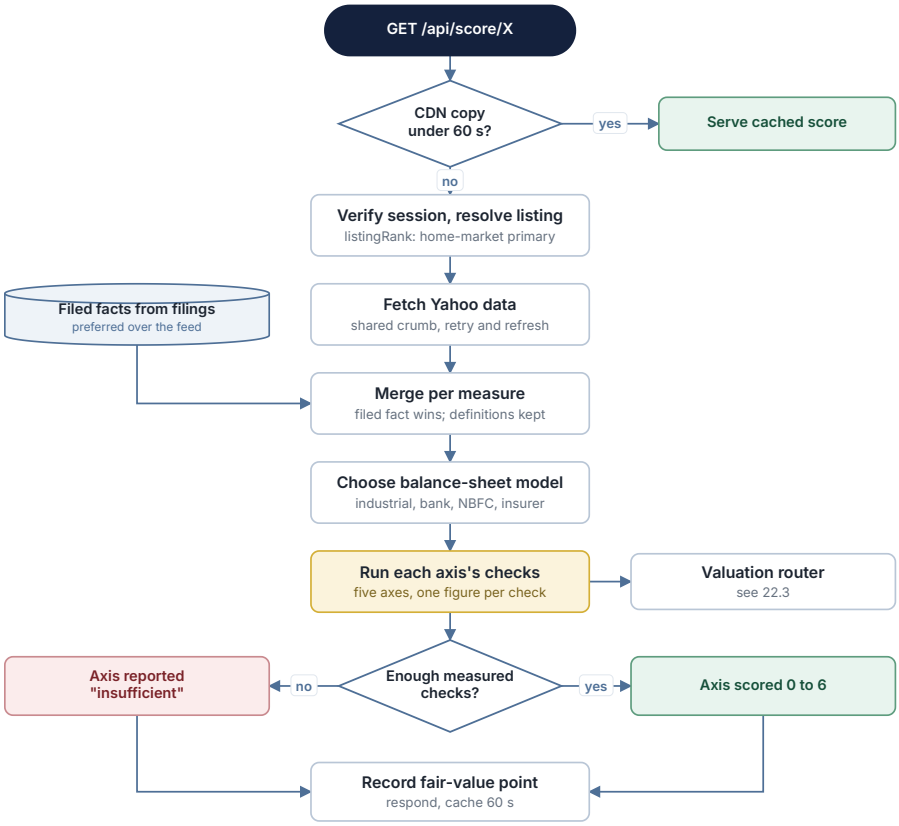


Figure 22.2 `/api/score/[symbol]` and `yahooFundamentals.ts`. The rule the flow protects: an axis without enough measured inputs is labelled insufficient rather than averaged over what is left.

22.3 Choosing a valuation

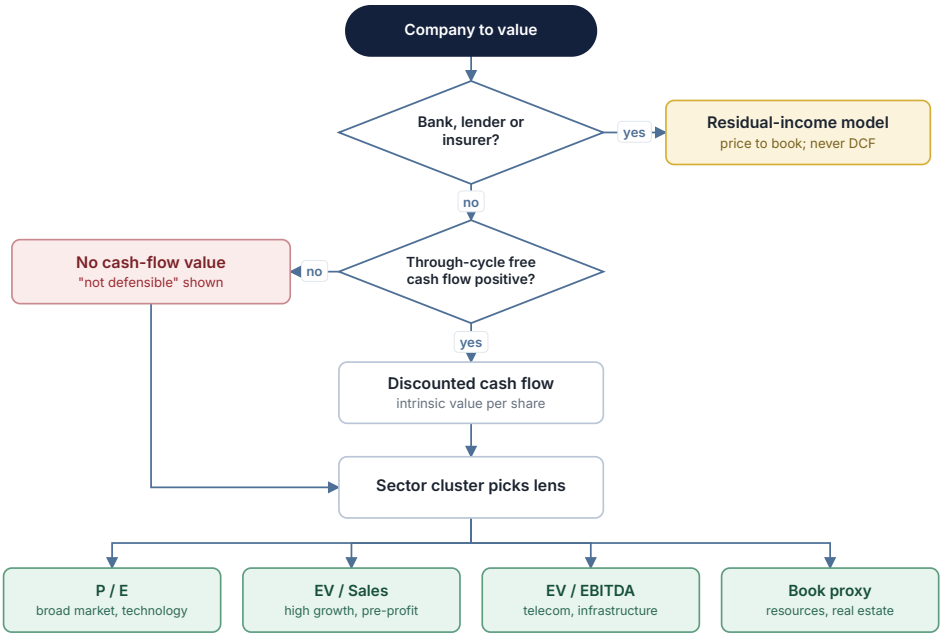


Figure 22.3 `valuationModel.ts`. Financial institutions are routed away from cash-flow models before any cash-flow figure is read; every other company gets a cash-flow value only when its free cash flow supports one, and a headline multiple chosen by sector.

22.4 Signing in

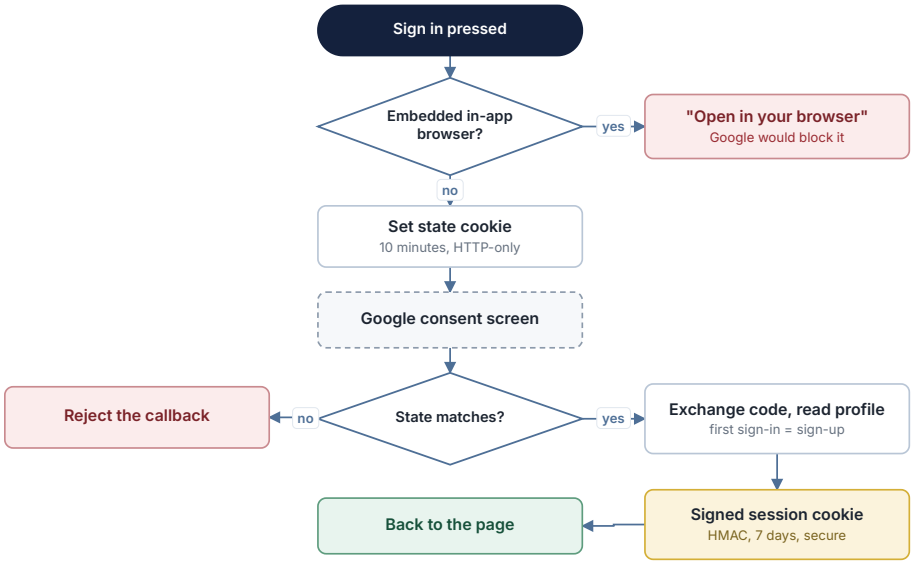


Figure 22.4 `/api/auth/login` and `/api/auth/callback/google`, with no authentication library. The session is checked again by every data route after this.

22.5 Paying for Pro

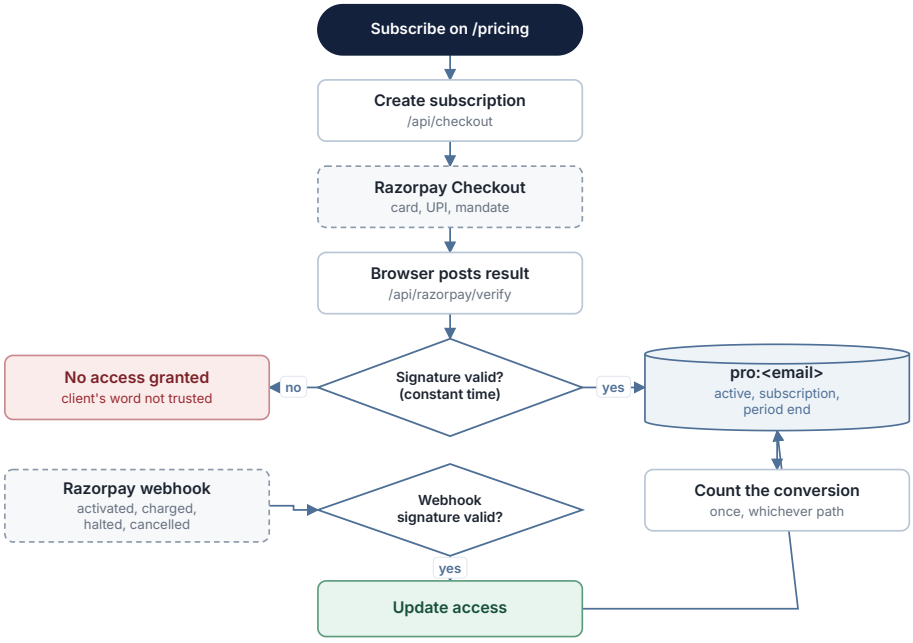


Figure 22.5 The subscription path through `razorpay.ts`, the verify route and the webhook. The one-time path differs only at the start (`/api/create-order`, then `/api/verify-payment`).

22.6 A headline becomes a card

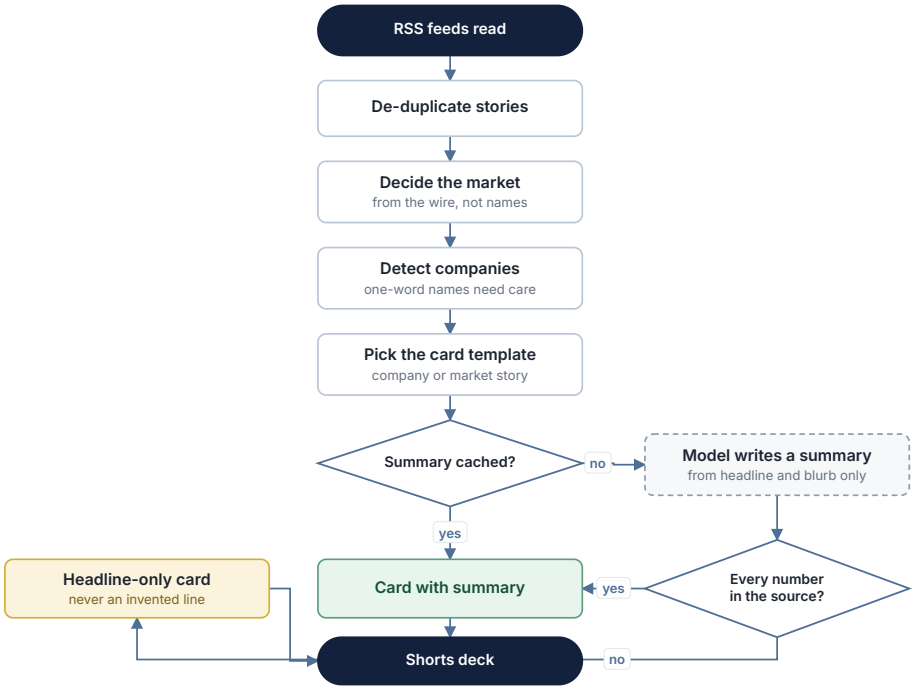


Figure 22.6 `newsShorts.ts`, `newsMarket.ts`, `tickerDetect.ts`, `storyTemplate.ts` and `newsSummary.ts`. A summary that introduces a figure its source did not contain is discarded, and the card falls back to the headline.

22.7 Publishing a post

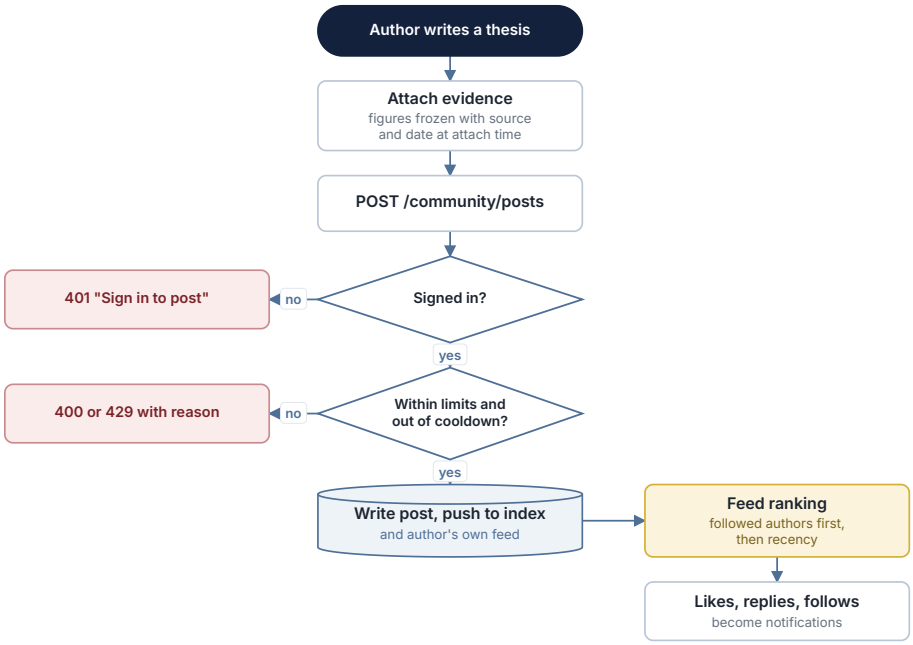


Figure 22.7 The community write path. The limits and cooldowns are pure functions in `communityPost.ts`; ranking happens on post ids before any post is fetched (`follow.ts`).

22.8 A nightly data job

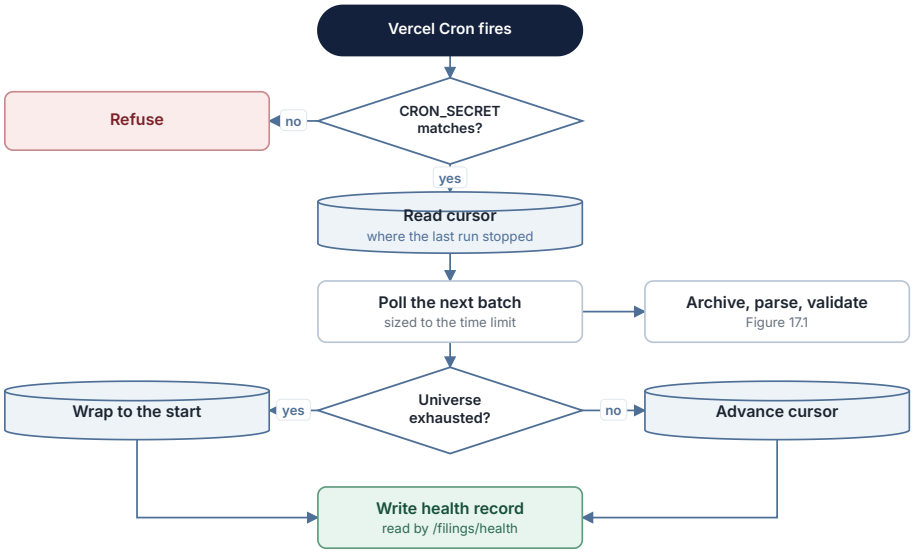


Figure 22.8 The shape shared by the filings jobs. Work is cut into batches that fit one serverless invocation, and a cursor in Redis lets the next night continue where this one stopped.

VI

PART VI

Evolution

The phases the product went through, reconstructed from 721 commits and described by what changed rather than when.

23 Development phases

23 Development phases

The history was replayed commit by commit and grouped into thirteen phases at the points where the direction of the work changed. Phases are numbered in the order they happened; no dates are given. For each phase the size of the codebase at its end was measured from the repository at that point.

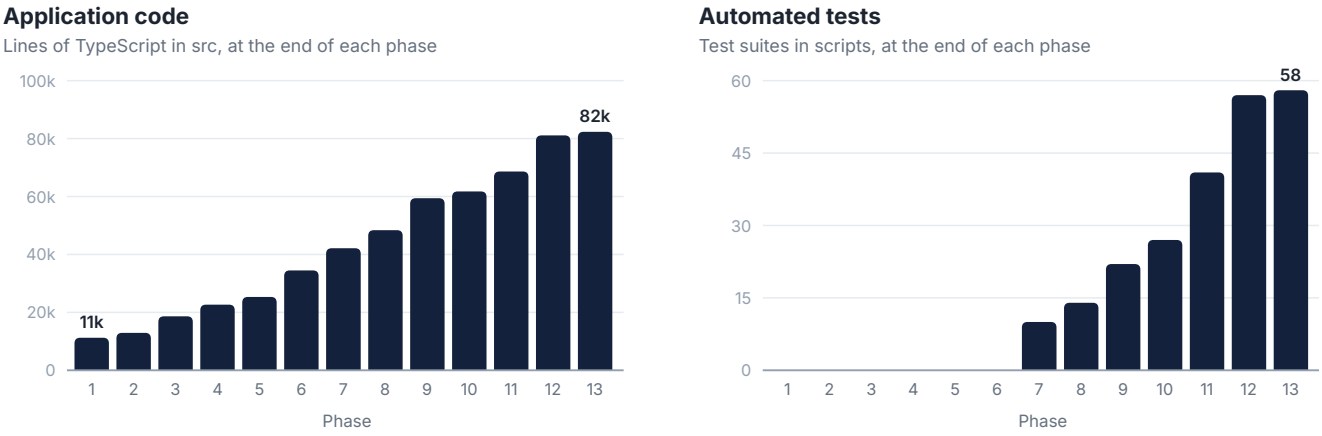


Figure 23.1 How the codebase grew. Two separate measures, drawn as two charts rather than on one doubled axis. Application code grew about sevenfold; automated tests did not exist until phase 7 and reached 58 suites by phase 13. Exact values are in Table 23.1.

#	PHASE	COMMITS	TS FILES	LINES	API ROUTES	TEST SUITES
1	A static prototype	45	89	11,253	17	0
2	Brand and payments	13	102	12,932	20	0
3	Breadth on the prototype	40	120	18,668	25	0
4	Real data	53	149	22,645	28	0
5	India coverage and security	58	159	25,345	30	0
6	A redesigned research page	81	193	34,460	37	0
7	Automation and reference data	55	228	42,128	47	10
8	Industry-aware analytics	65	239	48,371	50	14
9	Filings and a contextual assistant	77	285	59,391	61	22
10	AI assistants and data identity	68	294	61,767	61	27
11	An editorial product	67	323	68,627	68	41
12	Community and benchmarking	85	372	81,134	80	57
13	Depth and governance	14	378	82,313	81	58

Table 23.1 Size at the end of each phase. Lines and files count TypeScript under `src/app`, `src/components`, `src/lib` and `src/data`; a duplicate project copy briefly nested in the tree during phase 6 is excluded.

Phase 1 A static prototype · 45 commits

The repository began as an uploaded Next.js 14 project: an educational market-discovery prototype with a forced dark theme, inline-SVG charts and every number drawn from a static demo file. Its README promised no paid APIs, no broker connection and no buy or sell recommendations, a line the product still holds.

- Next.js App Router, TypeScript and Tailwind foundation
- Sixteen pages and seventeen API routes on demo data
- The research-signal framing: what is moving, why, and which stocks are affected

Phase 2 Brand and payments · 13 commits

The logo and icon set were settled and the first payment gateway was added. Pricing moved to the ₹49 launch offer (with ₹500 as the standard price), a launch-offer popup was mounted app-wide, and portfolios began fetching live quotes so holdings outside the demo set stopped showing blanks.

- Payment gateway and plan definitions
- Launch-offer popup
- Live quotes and sparklines for holdings

Phase 3 Breadth on the prototype · 40 commits

Features arrived quickly: funds and rare ETFs, notifications, insider data and competitor information, article viewing, a first community, notes, Razorpay, cash-flow-based valuation, and the Ideas section. Sign-in and access control were introduced, and a news-impact layer was introduced.

- ETF analysis, notifications, insider and competitor sections
- Community and Razorpay
- Cash-flow valuation and Investing Ideas
- Sign-in and access gating

Phase 4 Real data · 53 commits

The defining change of the project: the demo data was removed and every surface moved to live sources. Around it came compliance pages, search-engine optimisation, a light mode and mobile layouts, a reworked discounted-cash-flow model, a compare feature, a free trial, Vercel Analytics, and the first caching work to keep serverless compute in check.

- Removal of all demo data
- SEO, light mode, mobile adaptation
- DCF reworked repeatedly; Rare Finds and compare
- Edge caching for compute cost

Phase 5 India coverage and security · 58 commits

Indian insider trading was added from BSE and then NSE, with a scraping pipeline to reach exchange endpoints. Indian formatting arrived (lakhs instead of millions), country and currency detection, quarterly financials, an XIRR calculator, P/E history and editable holdings. The first security work landed: security headers, a content-security policy, DOMPurify and middleware.

- BSE and NSE insider pipelines
- Lakh and crore formatting, country and currency detection
- Security headers, CSP and middleware

Phase 6 A redesigned research page · 81 commits

The stock analysis page was rebuilt with a section navigation bar, new charts (debt to equity, cash flow, clickable chart events), news for Indian stocks and international coverage. A new landing page, a login page, an upgraded search and the first heatmap were added. A copy of the project that had been accidentally nested inside the source tree was found and removed.

- Stock analysis layout and navigation
- Landing and login pages
- Heatmap
- Clean-up of nested project copies

Phase 7 Automation and reference data · 55 commits

Ingestion moved off the request path: a GitHub Actions workflow pulled NSE insider disclosures into Redis, iterated until it read NSE's market-wide CSV reliably. Search was upgraded, executive profiles and management information were added, Taiwan insider data arrived, and the admin page was created. This is where the first test suites appear.

- Scheduled NSE insider ingestion in GitHub Actions
- Executive profiles, Taiwan insider data
- Admin page
- First test suites

Phase 8 Industry-aware analytics · 65 commits

The analytical core was rebuilt around the idea that one yardstick does not fit every business. Future Growth, the revenue-and-expenses flow (with bank, insurer and lender models), financial-health model selection, balance-sheet strength checks and listing ranking were each written as pure modules with their own tests.

- Future Growth and the revenue Sankey
- Balance-sheet models for banks, lenders and insurers
- Listing selection and grouping

Phase 9 Filings and a contextual assistant · 77 commits

Quantifi began reading companies' own filings: an XBRL parser, validation rules, an R2 archive, NSE and BSE adapters, RBI bank tables and official US bank balance sheets, followed by a securities master so the pipeline could cover the whole market. A DeepSeek-backed assistant appeared on the stock page. Pull requests became the unit of change.

- Filings pipeline with archive and validation
- RBI and FR Y-9C bank data
- Securities master
- Contextual AI on the stock page

Phase 10 AI assistants and data identity · 68 commits

The assistant gained a resizable drawer, real web search through DeepSeek's Anthropic-compatible endpoint, a portfolio-aware mode and an intent router, each with fixture-based tests. Fund identity and Indian ETF data were made reliable, and listing resolution was corrected for cases such as Take-Two and the German and Polish exchanges.

- Web-searching portfolio and stock assistants
- Fund identity and ETF reliability
- Exchange and listing corrections

Phase 11 An editorial product · 67 commits

The interface was given a hierarchy and a numeric voice: insider buying charted against selling, a lead-story news layout with a structured read, statements by quarter, an assumption sandbox on valuation, a funnel that counts every Pro conversion once, and the first Financial Shorts cards with real likes and comments.

- News read and Shorts cards
- Quarterly statements and valuation sandbox
- Conversion funnel and plan-claim tests

Phase 12 Community and benchmarking · 85 commits

The community was rebuilt around the post: theses with frozen evidence, a draft editor, follows, profiles, notifications and a verification tick. A formal design system was introduced in two named phases. Portfolios gained benchmark comparison against market indices and correct multi-currency arithmetic. The landing page became the scroll-driven ascent, and Financial Shorts became a full deck. The phase closed with a merged pull request covering the landing chart, Shorts and the community layer.

- Community: posts, profiles, follows, inbox
- Design system
- Portfolio alpha and FX
- Landing ascent and Financial Shorts

Phase 13 Depth and governance · 14 commits

The most recent work deepened analysis for non-financial companies (nine balance-sheet measures instead of three), derived missing ratios from statements rather than showing blanks, let users publish a portfolio by weight, and wrote down the compliance position.

- Nine-measure industrial balance sheet
- Derived ROE, ROA, P/S and PEG
- Published portfolios
- Compliance review

How the way of working changed

The history also shows the engineering practice maturing. The first phase was committed as file uploads; early messages were short labels (*UI changes*, *Fix*, *valuation fix*). From phase 9 work arrived through numbered pull requests, and in the later phases each commit message became a sentence describing the behaviour it changed (*Read the market as cards, one headline to a screen*; *A rupee is not a dollar, and four other things that were wrong*). Comments followed the same path: the modules written in the last five phases open with an account of the bug that motivated them and the rule that now prevents it, which is a large part of why this report could be written from the code alone. Tests arrived with the same shift: none for the first six phases, then a suite for nearly every rule that had once failed in production.

VII

PART VII

Quality

How the codebase is tested, what happened when every suite was run, and the individual test cases that guard the rules that matter most.

24 Testing approach

25 Test results

26 Test cases by domain

24 Testing approach

Quantifi has no test framework. It has 58 plain Node scripts in `scripts/`, each named `test-<subject>.mjs`, and an architecture that makes them sufficient: every rule worth testing lives in an *importless* TypeScript module, a file with no imports whose functions are pure. A test script compiles that one file with the TypeScript compiler into a temporary directory, imports the result, and drives it directly with no database, network, session or browser.

```
// scripts/test-public-portfolio.mjs (abridged)
const out = mkdtempSync(join(tmpdir(), "publicportfolio-"));
copyFileSync(join(root, "src/lib", "publicPortfolio.ts"), join(out, "publicPortfolio.ts"));
execFileSync("npx", ["tsc", "publicPortfolio.ts", "--module", "esnext", "--target", "es2022"], {
cwd: out });
const { toPublicHoldings } = await import(join(out, "publicPortfolio.js"));

let pass = 0, fail = 0;
const ok = (what, cond) => { if (cond) pass++; else { fail++; console.log(`  FAIL: ${what}`); } };

const { holdings } = toPublicHoldings([{ ticker: "TCS.NS", value: 400 }, ...]);
ok("a published holding carries ONLY a ticker and a weight",
   [...new Set(holdings.flatMap(Object.keys))].sort().join(",") === "ticker,weight");

console.log(`${pass} passed, ${fail} failed`);
process.exit(fail ? 1 : 0);
```

Conventions the suites share

CONVENTION	WHY IT MATTERS
Assertions are named as behaviour	A failure reads as the rule that broke: *an unknown pair is null, never 1*, not *assertEqual failed*
Each suite opens with the bug it exists for	The header records the production failure (HDFC Bank's valuation, a rupee added to a dollar, Take-Two in Austria) so the test's purpose survives its author
Fixtures come from production	Score projections, Taiwan datasets and web-search streams are tested against captured real payloads, not hand-written ones
Refusals are tested as carefully as answers	Many assertions check that something is *not* done: a figure not invented, a model not called, an id not accepted
Exit code is the verdict	Every suite exits non-zero on any failure, so a runner needs no parsing
No network	Upstreams are stubbed or replaced by fixtures; the whole set runs offline

Running the suites

Each suite runs on its own with `node scripts/test-<name>.mjs`. The whole set was run for this report with a loop over `scripts/test-*.mjs` after `npm ci`.

AN ENVIRONMENT TRAP, FOUND WHILE PREPARING THIS REPORT

The suites call `npx tsc`, which uses the TypeScript pinned in `package.json` (5.4.5) only when `node_modules` is installed. Without it, `npx` falls through to whatever global compiler exists. On a machine with TypeScript 6, 20 of the 58 suites stop before their first assertion with error TS5112, because TypeScript 6 refuses to compile named files while a `tsconfig.json` is in scope. That is an environment failure, not a code failure: after `npm ci` all 58 pass. One suite (`test-news-ticker-ranking`) already copies its sources out of the repository to avoid this, and the same pattern, or always running `npm ci` first, would make the others robust too.

What is and is not covered

LAYER	COVERAGE
Domain rules in `src/lib`	STRONG Scoring, valuation, filings, portfolio arithmetic, community rules, news guards, search and formatting
API routes	PARTIAL Symbol search is tested end to end with stubbed upstreams; other routes are covered only through the modules they call
React components	NONE No component or rendering tests; one suite checks a component's effect wiring by reading its source
Browser end to end	NONE No automated browser journeys (sign-in, checkout, posting)
Continuous integration	NONE The only workflow is data ingestion; tests and type checks run only when someone runs them

25 Test results

58 / 58	0	3,711	141 s
suites passed	failures	checks executed or declared	total run time, sequential

All 58 suites were run in a single sequential pass against commit `50aef04` using Node 22 and the project's pinned TypeScript 5.4.5. Every suite exited cleanly. Fifty-two suites report a count of named assertions, totalling 3,522. Six use Node's built-in `assert` module, which stops at the first failure and prints no count; for those the table shows the number of assertion call sites in the file (marked *), 189 in all, some of which run inside loops. Times include compiling the module under test.

SCORING AND VALUATION · 12 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`bank-valuation`	A bank never reaches the cash-flow model; residual income lands near the P/B read	85	2.1	PASS
`balance-sheet`	Sector checklists: thresholds, unavailable versus failed, sufficiency per domain	332	2.3	PASS
`financial-health`	Which of six balance-sheet models a company is read under	155	2.3	PASS
`future-growth`	Forecast arithmetic, horizon bookkeeping and refusals	95	3.9	PASS

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`income-flow`	Sankey conservation across eight reporting models	338	4.0	PASS
`statement-template`	Which statement lines a bank, insurer or industrial company has	40	2.2	PASS
`statement-period`	Annual and quarterly statement switching	19	0.1	PASS
`attachment-sources`	Score and figure projections, run against production payloads	139	2.5	PASS
`price-stats`	Price-performance panel arithmetic	32	2.1	PASS
`market-math`	Market-page aggregation (for example, why the mean of ratios is skewed)	92	4.0	PASS
`edgar-fundamentals`	SEC company-facts fallback: picking periods by end date, not fiscal year	14	3.7	PASS
`peer-selection`	Broad-sector names refused as competitors	5	3.9	PASS

FILINGS AND REFERENCE DATA · 10 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`filings`	XBRL parser safety, concept mapping, validation and company identity	142	4.2	PASS
`filings-e2e`	A filing goes in, a Balance Sheet Strength score comes out	142	2.0	PASS
`filing-facts`	Zero is a value, a negative is refused, conflicting columns are not published	29	2.1	PASS
`rbi-tables`	RBI bank-wise statistical tables	84	4.7	PASS
`securities-master`	Exchange master files: parsing and the NSE/BSE join	34*	1.7	PASS
`fund-identity`	Telling a fund from a company with only reliable fields	34*	1.5	PASS
`indian-etfs`	Indian ETF data reliability	22*	1.2	PASS
`officers`	Executive roster parsing and ordering	19	4.5	PASS
`exec-profile`	Matching an executive to a verified Wikidata profile	40	4.1	PASS
`currency`	Which currency a listing is quoted in (pence are not pounds)	17	0.1	PASS

SEARCH AND LISTINGS · 6 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`listing-rank`	Home-market primary ordinary share; grouping a company's listings	246	2.3	PASS
`search-rank`	Search ranking and word matching	27	4.6	PASS
`search-acceptance`	Keyword search stated as behaviour	22	4.1	PASS
`search-api`	Symbol search end to end with stubbed upstream indexes	21	3.8	PASS
`news-ticker-ranking`	The headline's company outranks an analyst attribution	7	2.1	PASS
`written-as-a-name`	A one-word company name must be written as a name	25	2.0	PASS

INSIDER DATA · 3 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`insider-volume`	Quarter bucketing; one insider filing five times is one insider	52	2.1	PASS
`taiwan-insider`	TWSE and TPEx dataset parsing against saved samples	79	3.8	PASS

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`pit-csv`	NSE insider CSV parsing outside the ingest script	25	0.1	PASS

PORTFOLIO · 6 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`portfolio-alpha`	Time-weighted returns against benchmarks	78	2.0	PASS
`portfolio-book`	One book, one set of weights for both halves of the page	29	2.2	PASS
`portfolio-fx`	Adding a multi-currency book; an unknown rate is null, never one	41	2.4	PASS
`portfolio-snapshot`	The holdings arithmetic the assistant may quote	25*	1.1	PASS
`public-portfolio`	A published portfolio carries only tickers and weights	36	2.1	PASS
`synced-envelope`	Reading on the server what the browser stored	10	2.1	PASS

COMMUNITY · 5 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`community-post`	What a post may contain; link parsing	71	2.0	PASS
`community-profile`	Handles, reserved names, links and profile fields	137	2.0	PASS
`community-image`	Upload byte sniffing and file naming	29	2.2	PASS
`community-inbox`	Which events become notifications, and how often	37	1.9	PASS
`follow`	Following and feed ordering	45	2.2	PASS

NEWS, SHORTS AND AI · 8 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`news-shorts`	Blurb detection and the numbers-in-source rule	96	2.3	PASS
`news-market`	Which market a story is about	42	2.4	PASS
`news-read`	The structured read beside a story	19	2.4	PASS
`news-summary-cache`	Caching of deck summaries	19	2.4	PASS
`story-template`	What a Financial Short shows	67	2.6	PASS
`market-summary`	The front page's market summary	72	2.2	PASS
`deepseek-websearch`	Web-search request shape and stream reader, against fixtures	39*	1.6	PASS
`ai-intent`	Which questions reach outside and which reach the reader's data	35*	0.1	PASS

PLATFORM AND BUSINESS RULES · 8 SUITES

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`admin-access`	Allowlists fail closed	34	4.4	PASS
`plan-claims`	The pricing page claims only what the code enforces	38	2.0	PASS
`pro-conversion`	What counts as a Pro conversion, counted once	32	2.1	PASS
`funnel-events`	The admin funnel's five steps	23	0.1	PASS
`shared-fetch`	Duplicate GETs collapse; per-account routes are never shared	17	2.7	PASS
`format`	How numbers, currencies and lakhs are written	68	2.2	PASS

SUITE	WHAT IT PROTECTS	CHECKS	SECS	RESULT
`heatmap-label`	Heatmap tile labels fit their tiles	55	2.1	PASS
`market-tape`	The landing chart behaves like a real price series	104	2.4	PASS

The five largest suites are `income-flow` (338), `balance-sheet` (332), `listing-rank` (246), `financial-health` (155), `filings-e2e` (142). Together they hold more than a third of all counted assertions, which matches where the codebase's own complexity sits.

26 Test cases by domain

The tables below list representative test cases, quoted exactly as they are named in the suites, with the rule each one protects. Identifiers are assigned here for reference; the suites themselves use the quoted names. Every case listed passed in the run reported in Chapter 25.

Scoring and valuation

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
SV-01	`bank-valuation`	*the cash-flow model was not called*	A bank's valuation never touches DCF
SV-02	`bank-valuation`	*the estimate is nowhere near the old ₹3,171.55*	The HDFC Bank regression cannot return
SV-03	`bank-valuation`	*an unknown sector is not assumed financial*	Missing data does not reroute a company
SV-04	`balance-sheet`	*an empty metric is unavailable*	Absence is reported, not scored
SV-05	`balance-sheet`	*zero is a real value, not a missing one*	A true zero still counts
SV-06	`balance-sheet`	*an unavailable check says why*	Every gap carries its reason
SV-07	`financial-health`	*a renamed US regional bank is still a US bank*	Model choice survives renames
SV-08	`financial-health`	*Insurance Brokers is not an insurer*	Industry strings are read precisely
SV-09	`statement-template`	*gross profit is not offered to a bank*	No meaningless lines on a bank's statement
SV-10	`income-flow`	*no link is negative*	The Sankey never draws a negative width

Filings

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
FI-01	`filings`	*an entity bomb is refused outright*	Malicious XML cannot exhaust the parser
FI-02	`filings`	*an empty document is an error, not a crash*	Bad input fails cleanly
FI-03	`filing-facts`	*grossNpa of -1 is refused as negative, not as zero*	A bad figure is refused, not zeroed
FI-04	`filing-facts`	*taxExpense of 0 survives*	Legitimate zeros are kept
FI-05	`filing-facts`	*neither column is published*	Two conflicting values publish neither
FI-06	`filing-facts`	*the reason names both values*	A refusal is explainable

Listings and search

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
LS-01	`listing-rank`	*NYSE Arca is not NYSE*	Exchange names are not matched loosely
LS-02	`listing-rank`	*Stuttgart by suffix alone*	Venue is read from the ticker suffix
LS-03	`written-as-a-name`	*Situational Awareness is not a company called Awareness*	Prose is not a company name
LS-04	`written-as-a-name`	*the central bank is not Central anything*	Common words stay words
LS-05	`currency`	*GBp is pence, not pounds*	London prices are not overstated a hundredfold
LS-06	`news-ticker-ranking`	*headline company beats and removes analyst attribution*	The story's subject leads

Portfolio

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
PF-01	`portfolio-fx`	*an unknown pair is null, never 1*	A rupee is never assumed to be a dollar
PF-02	`portfolio-fx`	*a zero rate is not a rate*	Corrupt rates are refused
PF-03	`portfolio-fx`	*a tie goes to the first seen, so it does not wander*	Display currency is stable
PF-04	`portfolio-alpha`	*the intersection, not the union*	Benchmark and book share the same days
PF-05	`public-portfolio`	*a published holding carries ONLY a ticker and a weight*	No amount can leak
PF-06	`public-portfolio`	*ten times the money produces an identical publication*	Weights reveal nothing about size
PF-07	`public-portfolio`	*two lots of one company are one line*	Lots merge before publishing

Community

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
CM-01	`community-image`	*an SVG is refused, script or no script*	No scriptable images
CM-02	`community-image`	*PHP is refused*	Declared type cannot smuggle code
CM-03	`community-image`	*an ELF binary is refused*	Executables are rejected by their bytes
CM-04	`community-profile`	*case is not identity*	Handles cannot be duplicated by case
CM-05	`community-profile`	*starting with a digit*	Handle shape rules hold
CM-06	`follow`	*a key-traversing string is not*	Ids cannot escape their key space
CM-07	`community-inbox`	*somebody else liking it is news*	Notifications fire on the right events

News and AI

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
NA-01	`news-shorts`	*a summary may not add a number*	Summaries cannot invent figures
NA-02	`news-shorts`	*the echoed headline is not a blurb*	A headline is not mistaken for an article
NA-03	`news-market`	*a rupee figure is Indian*	Market detection reads currency

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
NA-04	`news-market`	*a story naming no venue names no market*	No market is guessed
NA-05	`story-template`	*a question is an argument, not an event*	Opinion pieces get the right card

Platform

ID	SUITE	TEST CASE (AS NAMED)	RULE PROTECTED
PL-01	`admin-access`	*a star does not admit everyone*	No wildcard in an allowlist
PL-02	`admin-access`	*an empty email is never admitted*	Missing identity is refused
PL-03	`plan-claims`	*no cent sign anywhere in the plan*	The ¢0.55 pricing bug cannot return
PL-04	`plan-claims`	*at least one row is genuinely Pro only*	The plan comparison stays meaningful
PL-05	`shared-fetch`	*ten components, one request*	Request de-duplication works
PL-06	`shared-fetch`	*a per-account route is never shared*	One reader never sees another's data
PL-07	`format`	*NaN is not*	Invalid numbers are never printed

READING THE PATTERN

Roughly half of these cases assert that something does *not* happen. That is the signature of a codebase whose worst failures are silent: a number that looks plausible and is wrong, rather than a crash. The suites are written to catch exactly that class of error.

VIII

PART VIII

Assessment

What the codebase does well, where the risks are, and the order in which to address them. A glossary closes the report.

27 Strengths, risks and recommendations

A Glossary

27 Strengths, risks and recommendations

Strengths

- **Correctness culture.** The rule that an unmeasured figure is shown as unavailable is applied consistently across scoring, valuation, filings, currency and news. This is unusual and valuable in a financial product, where a plausible wrong number does more harm than a blank.
- **Domain depth.** Separate balance-sheet, income-statement and valuation models for banks, lenders, insurers and industrial companies; home-market listing selection; filings preferred over vendor data. These are the parts competitors usually get wrong.
- **Testable architecture.** Importless pure modules for every decision make 58 suites and about 3,700 checks possible with no framework and no network.
- **Lean dependencies.** Seven production packages; authentication, payments, Redis and S3 signing written against the APIs directly.
- **Security and privacy by construction.** Nonce-based CSP, signed sessions, encrypted user records, one-way community identity, weights-only publishing, fail-closed allowlists, byte-level upload checks.
- **Self-documenting code.** Modules explain the failure that motivated them, which makes the system unusually easy to hand over.

Risks

RISK	SEVERITY	DETAIL
No continuous integration	HIGH	Tests, type checks and lint run only by hand. A regression can reach production unnoticed, and the build-breaking unused-import error seen during this project's history would have been caught before deploy.
Framework advisory	HIGH	npm reports a published security advisory for the pinned Next.js 14.2.15. A patched 14.2 release exists.
Data-source terms	HIGH	Prices and much of the fundamentals come from sources whose free terms contemplate personal use, while the product is paid. This is a commercial and licensing matter rather than a code defect, and is addressed in the separate compliance review.
Single unofficial upstream	MEDIUM	Much of the analysis depends on Yahoo Finance's unofficial endpoints, which have already changed behaviour once (empty statement modules). The filings pipeline reduces but does not remove this dependency.
Job routes fail open	MEDIUM	Scheduled-job routes accept any caller when `CRON_SECRET` is unset, and accept it in the query string. Safe only while the variable stays configured.
Public-prefixed fallbacks	MEDIUM	`NEXT_PUBLIC_FMP_API_KEY` and `NEXT_PUBLIC_PRO_EMAILS` would be compiled into the browser bundle if ever set.
Test toolchain fragility	MEDIUM	Twenty suites depend on `npx` resolving the pinned compiler (Chapter 24).
No UI or end-to-end tests	MEDIUM	Sign-in, checkout and posting are untested as journeys.
Dead payment dependency	LOW	`stripe` and `src/lib/stripe.ts` are unused.
Notes are device-local	LOW	Stock notes live only in the browser, unlike portfolios and the watchlist.

Recommended order of work

#	ACTION	EFFORT
1	Add a GitHub Actions workflow that runs `npm ci`, `tsc --noEmit`, `next lint` and all 58 suites on every push and pull request	Half a day
2	Upgrade Next.js to the patched 14.2 release and confirm the build	Under a day
3	Make the job routes fail closed and header-only; delete the two `NEXT_PUBLIC_` fallbacks and the unused Stripe client and package	A few hours
4	Make the test harness independent of the global compiler (copy-out pattern or a pinned `tsc` path)	A few hours
5	Add browser journeys for sign-in, the free-limit wall, checkout and posting (Playwright is already available in most CI images)	Two to three days
6	Move priced market data to a commercially licensed feed behind the existing adapters; extend the filings pipeline for fundamentals	Weeks, plus subscription cost
7	Sync stock notes through the same encrypted store as portfolios	Under a day

> OVERALL
>
> The codebase is well engineered for its size and stage: deliberate in its architecture, careful with correctness and privacy, and unusually well explained. Its main gaps are in automation around the code (continuous integration, dependency updates, journey tests) rather than in the code itself, and they are inexpensive to close.

A Glossary

TERM	MEANING IN THIS CODEBASE
App Router	Next.js's file-based routing in `src/app`; a folder with `page.tsx` is a page, with `route.ts` an API endpoint
Axis	One of the five dimensions of the Quantifi Score, scored from a list of checks
Check	One threshold on one measured figure, with its definition and, if unavailable, a reason
Crumb	The token Yahoo requires alongside a cookie before answering statement requests
CSP nonce	A random value generated per request; only scripts carrying it may run
DCF	Discounted cash flow: valuing a business by the present value of its future free cash flow
Edge middleware	Code that runs at Vercel's network edge before a page or route is invoked
Explorer, Pro	The free plan and the paid plan
FR Y-9C	The consolidated financial statements US bank holding companies file with the Federal Reserve
HyperLogLog	A probabilistic counter that estimates distinct visitors without storing who they were
Importless module	A TypeScript file with no imports, so a test can compile and run it in isolation
ISIN, CIN	Permanent identifiers for a security and for an Indian company; used instead of tickers to key filings
KV	The Upstash Redis store, reached over its REST API

TERM	MEANING IN THIS CODEBASE
NBFC	Non-banking financial company: an Indian lender that cannot take most kinds of deposit
NPA	Non-performing asset: a loan in default; net NPA is after provisions
PIT disclosure	An insider-trading disclosure under SEBI's Prohibition of Insider Trading regulations
R2	Cloudflare's S3-compatible object storage, used as the filings archive
Residual income	A valuation that values a firm by the returns it earns above its cost of equity; used for financial institutions
s-maxage, SWR	CDN cache lifetime, and the window in which a stale copy may be served while one refresh runs
Securities master	The list of listed companies the filings pipeline is allowed to look for
Shared fetch	The client layer that merges identical in-flight GET requests
Snowflake	The five-axis radar drawing of a company's Quantifi Score
Time-weighted return	A return that ignores the timing and size of deposits, used for benchmark comparison
XBRL	The tagged machine-readable format in which companies file financial statements

B Build prompts

This appendix sets out the prompts that build Quantifi, phase by phase, in the same order as Chapter 23. Each prompt names a goal, lists the requirements that define it, states the rules it must not break, and ends with a check that shows whether it worked and the code that implements it.

To work through the sequence, start a new session, paste the project brief (prompt 0) once, then run prompts 1 to 13 in order, building and testing after each. A prompt can be split if a smaller step is wanted. Phase 12 is two prompts because the community layer and the portfolio layer are independent.

Prompt index

#	PHASE	WHAT THE PROMPT ASKS FOR
0	**Shared context**	Project brief, given once at the start
1	**A static prototype**	Foundation and demo-data prototype
2	**Brand and payments**	Pricing, checkout and the launch offer
3	**Breadth on the prototype**	Quantifi Score, screener, accounts
4	**Real data**	Replace demo data with live sources
5	**India coverage and security**	Indian data, formatting and hardening
6	**A redesigned research page**	Stock page, search and heatmap
7	**Automation and reference data**	Scheduled ingestion, Redis, first tests
8	**Industry-aware analytics**	One yardstick per kind of business
9	**Filings and a contextual assistant**	Read the company's own filings
10	**AI assistants and data identity**	Web-searching and portfolio-aware assistants
11	**An editorial product**	News reads, insider charts, Shorts
12a	**Community and benchmarking**	Community of theses
12b	**Community and benchmarking**	Honest portfolios and a design system
13	**Depth and governance**	Nine-measure balance sheet and published portfolios

The prompts

0 Project brief

SHARED CONTEXT

You are helping me build Quantifi, a market-discovery and portfolio-intelligence web app for young retail investors. India is the home market; the US, Taiwan and other major exchanges are extra coverage. Its promise: find what is moving, why it is moving, and which stocks are affected.

REQUIREMENTS

1. Stack: Next.js 14 App Router, TypeScript in strict mode with an @/* alias to src/, and Tailwind CSS, deployed on Vercel as one serverless codebase with no separate backend.
2. Dependencies: keep production packages few. Call REST APIs with fetch and use Node's built-in crypto instead of vendor SDKs.
3. Look: dark and mobile-first, on an ink background (#05070D to #1A2234), a gold accent (#E9B872), a teal data accent (#4FD1C5), green (#34D399) for up and red (#FB7185) for down. Fonts: Inter, Space Grotesk and JetBrains Mono through next/font.
4. Structure: routes stay thin (check identity, validate input, call src/lib, return). Every decision rule lives in a small pure module that can be tested with no database and no network.

RULES

- Never invent a number. If a figure cannot be measured, show it as unavailable, never as zero, and never present an estimate as a fact.
- The product is educational: no buy, sell or hold recommendations, and a disclaimer on every analysis surface.
- Work one step at a time, tell me what you changed, and stop for my review at the end of each prompt.

CHECK Paste this once at the start of the project and again at the start of any new session.

EVIDENCE `README.md` `tailwind.config.ts` `src/app/layout.tsx`

1 Foundation and demo-data prototype

PHASE 1 · A STATIC PROTOTYPE

Create the project foundation and a complete prototype that runs on static demo data, so the product idea can be judged before any data source is connected.

REQUIREMENTS

1. Create a Next.js 14 App Router app in TypeScript and Tailwind with a src/ directory. Force a dark theme through the html element and tailwind darkMode: "class", with no light-theme path, so a system preference cannot break the UI.
2. Build a sticky navbar with a mobile menu, a footer with the disclaimer, and an auto-scrolling ticker strip of market movers that loops seamlessly.
3. Build the research pages: home, portfolio, ideas, news, insider-activity, explore, screener, famous-takes, stock-analysis and watchlist, plus pricing and currencies pages.
4. Put all data in one typed file, src/data/demo.ts: a stock universe (ticker, name, region, exchange, sector, price, day change, market cap, sparkline series), holdings, trading ideas in ten themes (AI Infrastructure, Recent IPOs, Global Space Race Stocks, Undervalued Companies, Dividend Powerhouses, Insider Activity, Nuclear Energy, Autonomous Vehicles, Cybersecurity, Space Economy), news items, eight kinds of insider event, famous investor takes, and formatting helpers (fmtPrice, fmtPct, fmtCompact).
5. Build shared primitives in Cards.tsx: GlassCard, ChangePill, TickerChip, Tag, StatTile, and inline-SVG Sparkline, Donut and BarMeter, with no chart library.
6. Build the product's signature component, the Impact Chain. It shows how one news event ripples outward: event, then direct stocks, then peers, then the sector, then ETFs and indices, each with an up, down or flat direction.
7. Define the palette once in tailwind.config.ts as ink, gold, teal, up and down, and add glass-panel utilities, a faint grid backdrop and keyframes for the marquee, floating cards and pulsing dot in globals.css.

RULES

- No paid APIs, no broker connection, and no recommendations. Say so in the README.
- Editing demo.ts alone must be able to reshape the whole prototype.

CHECK Every page renders at phone width, and editing demo.ts changes the whole prototype.

EVIDENCE `src/data/demo.ts` `src/components/quantifi/Cards.tsx` `tailwind.config.ts` `src/app/layout.tsx` `README.md`

2 Pricing, checkout and the launch offer

PHASE 2 · BRAND AND PAYMENTS

Add a pricing page and a payment gateway that stays dormant until keys exist, then settle the brand and the launch offer.

REQUIREMENTS

1. Define plans in one file, src/data/plans.ts: Explorer (free), Plus (₹499 a month) and Pro (₹1,299 a month), each with an id, name, display price, period, tagline, feature list, call-to-action and, for paid plans, the name of the environment variable that holds its Stripe Price ID.
2. Build /pricing from that file (PricingPlans.tsx) with the middle plan highlighted.
3. Add POST /api/checkout. Validate the plan id against plans.ts, read the Price ID from the environment, and create a Stripe Checkout session in subscription mode with promotion codes allowed and success and cancel URLs built from the request origin or NEXT_PUBLIC_BASE_URL. Add /billing/success and /billing/cancel pages.
4. Add POST /api/webhook. Read the raw body, verify the stripe-signature header against STRIPE_WEBHOOK_SECRET, and switch on checkout.session.completed, customer.subscription.updated and customer.subscription.deleted, leaving access-granting as a marked TODO until a user system exists.
5. Create the Stripe client in src/lib/stripe.ts so it is null when no key is set.
6. Then simplify to one paid plan with a ₹49 launch price (standard price ₹500) and mount a launch-offer popup across the app that appears once per session. Settle the logo and icon set, and make portfolios fetch live quotes so holdings outside the demo set stop showing blanks.

RULES

- Without keys the app must build and run. Checkout answers 503 with a plain message, the webhook answers 503, and a missing Price ID answers 500 naming the variable to set.
- No secret or price ID is ever committed; everything comes from environment variables.

CHECK With no Stripe key, /api/checkout answers 503 with a clear message instead of failing.

EVIDENCE `src/data/plans.ts` `src/app/api/checkout/route.ts` `src/app/api/webhook/route.ts` `src/lib/stripe.ts` `PricingPlans.tsx`

3 Quantifi Score, screener, accounts

PHASE 3 · BREADTH ON THE PROTOTYPE

Add depth to the research side and the first account system, building each feature as a checklist the reader can inspect rather than a bare number.

REQUIREMENTS

1. Build the Quantifi Score as a typed model. Five axes (Value, Future Growth, Past Performance, Financial Health, Dividends), each scored 0 to 6 from a list of named pass-or-fail checks, plus a fair-value estimate with its method and note, a list of rewards and a list of risk flags. Provide overallScore() and valuationGapPct() helpers.
2. Render the score as a radar snowflake (CompanySnapshot.tsx) that lists every check and whether it passed.
3. Build the screener (Screener.tsx): filter by region, sector, minimum score and undervalued only, sorted by total score, with change pills and ticker chips.
4. Build the stock explorer (StockExplorer.tsx) with a search box, quick-pick tickers and a chart-engine toggle, and a currency tool (CurrencyTools.tsx) with a converter, a swap button and a rates board with removable currencies.
5. Build manual portfolios in the browser (usePortfolios.ts): several named portfolios, holdings of ticker, shares and average cost, live totals, gain and loss, and allocation. Save to localStorage under a versioned key and seed a sample portfolio on first run.
6. Add fund and ETF analysis, notifications, insider and competitor sections, private notes, a first community feed, cash-flow-based valuation and an Ideas section.
7. Add Google sign-in written directly against Google's OAuth endpoints with an HMAC-signed cookie, edge middleware that gates private pages, and Razorpay for payments with server-side signature checks.

RULES

- A signed-out visitor is redirected away from gated pages; a forged cookie must be rejected by signature, not trusted.
- The screener and the stock page must use the same total score.

CHECK The score and the screener use the same total, and signed-out visitors are redirected away from gated pages.

EVIDENCE `src/data/demo.ts` `CompanySnapshot.tsx` `Screener.tsx` `StockExplorer.tsx` `CurrencyTools.tsx` `usePortfolios.ts`

4 Replace demo data with live sources

PHASE 4 · REAL DATA

Remove every piece of demo data so that each surface shows live, public data, and make every data route fail soft.

REQUIREMENTS

1. GET /api/quote/[symbol]: validate a ticker and return its price, currency and name from Yahoo's chart endpoint (range 5d), falling back to Stooq end-of-day data, and answering valid: false for a junk ticker. Cache for 120 seconds.
2. GET /api/timeseries/[symbol]: daily closes for the chart from Yahoo, then Stooq, returned as { time: YYYY-MM-DD, value } points with price, change and currency metadata.
3. GET /api/fx?from=&to=: a rate from Frankfurter (ECB data, keyless) first, then Yahoo's currency pair, with identity for equal currencies. Cache for 10 minutes.
4. GET /api/score/[symbol]: real fundamentals from Yahoo's quoteSummary after its cookie-and-crumb handshake. Turn P/E, PEG, margins, ROE, growth, current ratio, debt to equity, cash and debt, dividend yield and payout ratio into the five score axes through one axis() helper, so an axis is the share of its checks that passed, scaled to 0 to 6. Return nothing for ETFs and indices, which have no company fundamentals.
5. GET /api/fundamentals/[ticker]: latest annual revenue, net income, assets, liabilities and cash from SEC EDGAR for US filers, through a polite fetcher (src/lib/ingest/politeFetch.ts) that sets a declared User-Agent, a 10-second timeout and a cache, and a cheerio-based scrapeHtml() extension point.
6. Replace the embedded chart widget with TradingView's open-source lightweight-charts (PriceChart.tsx), themed to the app, with range buttons.
7. Add CDN caching with s-maxage and stale-while-revalidate, so serverless compute stays cheap. Add privacy, terms and refund pages, search-engine metadata, a light mode and mobile layouts.

RULES

- Every route fails soft: if a source is down, the response says unavailable. It never returns a made-up value.
- Only point ingestion at sources whose terms allow it. Check robots.txt and terms first, and state personal-use limits in the README.

CHECK A junk ticker is reported as not valid and can never be added to a portfolio.

EVIDENCE `api/quote` `api/timeseries` `api/fx` `api/score` `api/fundamentals` `lib/ingest/edgar.ts` `lib/ingest/politeFetch.ts` `lib/yahooFundamentals.ts` `PriceChart.tsx`

5 Indian data, formatting and hardening

PHASE 5 · INDIA COVERAGE AND SECURITY

Make India a first-class market, then harden the app before any more features are added.

REQUIREMENTS

1. Add Indian insider-trading disclosures (SEBI PIT filings): BSE first, then NSE, through a scraping pipeline with a proxy fallback for hosts that block cloud servers.
2. Format money the Indian way, in lakhs and crores, in one file, src/lib/format.ts, and detect each listing's country and currency (for example from the .NS and .BO suffixes). Keep US formatting for US listings.
3. Add quarterly financials, P/E history and editable holdings recorded as lots, and an XIRR calculator that solves dated cash flows with a bisection-then-Newton solver.
4. Add security headers: HSTS, X-Content-Type-Options, Referrer-Policy and Permissions-Policy.
5. Generate a Content-Security-Policy per request in middleware with a fresh nonce, so only scripts carrying that nonce may run. Add DOMPurify for any HTML that has to be rendered.

RULES

- Formatting rules live in one file and every screen uses it; no screen formats a currency on its own.
- Never render untrusted HTML without sanitising it first.

CHECK ₹12,345,678 is shown as ₹1,23,45,678 for an Indian listing and with ordinary grouping for a US one.

EVIDENCE `src/lib/format.ts` `src/middleware.ts` `public/ (vendored DOMPurify)`

6 Stock page, search and heatmap

PHASE 6 · A REDESIGNED RESEARCH PAGE

Rebuild the stock analysis page so a reader can understand one company from a single scrolling page, and improve discovery.

REQUIREMENTS

1. Assemble the stock page as one long page in StockExplorer, ordered as a reader would reason: price, score, valuation, statements, peers, ownership, insider trades and news.
2. Add a sticky section rail that lists only the sections that actually rendered. A section with no data is omitted, not shown empty.
3. Add a debt-to-equity history chart, a cash-flow chart, and chart markers for dated corporate events that the reader can click.
4. Add news for Indian stocks and for international markets, and international coverage on the page.
5. Build a landing page for visitors, a login page, and a full-panel command search opened with Ctrl or Cmd K that shows recent searches and groups a company's listings together.
6. Add the first market heatmap. Find and remove any copy of the project accidentally nested inside the source tree.

RULES

- The rail and the page are built from the same list of rendered sections, so the rail can never link to something missing.

CHECK A security with no data for a section shows no empty box for it, and the rail never links to a missing section.

EVIDENCE `StockExplorer.tsx` `StockSectionNav` `CommandSearch.tsx` `DebtEquityHistory.tsx` `MarketHeatmap.tsx`

7 Scheduled ingestion, Redis, first tests

PHASE 7 · AUTOMATION AND REFERENCE DATA

Move slow data collection off the request path, add a store for it, and start testing the rules that have already gone wrong.

REQUIREMENTS

1. Add a GitHub Actions workflow that runs on weekdays, downloads NSE's market-wide insider-trading CSV and writes one record per symbol to Redis. Support a manual run with a look-back window and a reset flag.
2. Write a small Upstash Redis client over its REST API, with pipelining, in src/lib/kv.ts. Use no SDK.
3. Add executive profiles matched to Wikidata and Wikipedia (education, career and portrait), and Taiwan insider data from the TWSE and TPEx open-data sets.
4. Add an /admin page for staff, gated by a fail-closed allowlist: an empty list admits nobody.
5. Start testing: write plain Node scripts named scripts/test-<subject>.mjs, each compiling one importless TypeScript module and exiting non-zero on any failure. Cover the first ten rules.
6. Add /api/insider/status so an empty result can be diagnosed.

RULES

- A scheduled job processes a bounded batch and stores a cursor, so a re-run resumes instead of starting again.
- Tests must run with no framework and no network.

CHECK When insider data is missing, /api/insider/status says which of the four failure points stopped it.

EVIDENCE `.github/workflows/` `src/lib/kv.ts` `scripts/test-*.mjs` `/admin`

8 One yardstick per kind of business

PHASE 8 · INDUSTRY-AWARE ANALYTICS

Rebuild the analytics so that one yardstick is never applied to every business, with each rule a pure module and its own test.

REQUIREMENTS

1. financialHealth.ts: choose one of six balance-sheet models (industrial, bank, US bank, NBFC, life insurer, general insurer) from the company's industry.
2. balanceSheet.ts: hold each model's checklist. The industrial list covers leverage, liquidity and cash. A check that cannot be measured is excluded from the count, and an axis with too few measurable checks is marked insufficient rather than scored.
3. valuationModel.ts: send banks, lenders and insurers to a residual-income model and never to discounted cash flow. For everyone else choose a headline multiple by sector (for example P/E for software, EV/EBITDA for telecom).
4. incomeFlow.ts: draw a revenue Sankey that conserves every flow and shows any shortfall as an explicit Other.
5. futureGrowth.ts: state the horizon it actually used instead of implying one.
6. listingRank.ts: choose the issuer's home-market primary ordinary share, never a foreign receipt, and group listings of one company.
7. Rename the five score axes in plain language: Valuation Comfort, Growth Durability, Profitability Quality, Balance Sheet Strength and Capital Allocation.

RULES

- Each module has no imports, so a test can compile and run it alone.
- Each module's opening comment states the failure that motivated it and the rule that now prevents it.

CHECK The cash-flow model is never called for a bank, and HDFC Bank's value is nowhere near the old ₹3,171.55.

EVIDENCE `financialHealth.ts` `balanceSheet.ts` `valuationModel.ts` `incomeFlow.ts` `futureGrowth.ts` `listingRank.ts`

9 Read the company's own filings

PHASE 9 · FILINGS AND A CONTEXTUAL ASSISTANT

Teach Quantifi to prefer a company's own filed numbers over a vendor's, and add an assistant that answers from the page it sits on.

REQUIREMENTS

1. In src/lib/filings add an XBRL parser, validation rules (period, scale and units, comparatives, sanity checks) and adapters for NSE and BSE results, the RBI's bank-wise tables and the US FR Y-9C bank balance sheets.
2. Archive each original document in Cloudflare R2 under its content hash, signing requests with an in-house AWS Signature V4 and no SDK.
3. Map facts to metrics with industry-aware concept tables. Key companies by ISIN or CIN, never by ticker.
4. Rank extraction methods so the better one wins: XBRL, then regulator table, HTML, PDF text, OCR, and a language model only as a last resort.
5. Store a fact that fails validation as rejected, with its reason. Never turn it into zero.
6. Run nightly cron jobs in cursor-sized batches that fit a serverless time limit, build a weekly securities master so the pipeline covers the whole market, and add /api/filings/health.
7. Add a DeepSeek-backed assistant to the stock page that answers from a financial snapshot of the page's own data.

RULES

- Cron routes accept only the scheduler's secret and fail closed.
- The assistant must say when the page data does not contain the answer.

CHECK A filing goes in and a Balance Sheet Strength score comes out, end to end.

EVIDENCE `src/lib/filings/` `src/lib/filings/r2.ts` `/api/cron/filings-*` `/api/filings/health` `StockAiAssistant.tsx`

10 Web-searching and portfolio-aware assistants

PHASE 10 · AI ASSISTANTS AND DATA IDENTITY

Extend the assistant with web search and portfolio awareness, and make instrument identity reliable.

REQUIREMENTS

1. Give the assistant a resizable drawer and real web search through DeepSeek's Anthropic-compatible Messages endpoint, in deepseekWebSearch.ts. It must use that endpoint's own path, headers, tool format and stream events.
2. Add a portfolio assistant that reads the selected portfolio, runs the screener and searches the web.
3. Build the portfolio snapshot in the browser (portfolioSnapshot.ts) and send it with the question, so the server stores none of it. Cap it at 40 positions.
4. Add an intent router: questions about the reader's own holdings use the attached portfolio, and questions about the world use bounded web research.
5. Make fund identity and Indian ETF data reliable. Fix listing resolution for Take-Two (it resolved to Austria) and for the German and Polish exchanges.
6. Write fixture-based tests for each of these, so no test calls a live model.

RULES

- Web research is bounded in the number of searches and the time taken.
- The assistant reasons from the data it is given and never invents a holding.

CHECK "How much of my book is this stock?" uses the portfolio, and "what did the Fed decide?" goes to web research.

EVIDENCE `deepseekWebSearch.ts` `portfolioAi.ts` `portfolioSnapshot.ts` `/api/ai/stock` `/api/ai/portfolio`

11 News reads, insider charts, Shorts

PHASE 11 · AN EDITORIAL PRODUCT

Give the interface a hierarchy and a numeric voice, and make news something the reader can act on without being misled.

REQUIREMENTS

1. Chart insider buying against selling by quarter (insiderVolume.ts). One person filing five times counts as one insider.
2. Give the news page a lead story and a list. Opening a story writes a structured read: what changed, why it matters, the causal path, the companies affected, what to watch, and what would undercut the read. The last line must always be present.
3. Add statements by quarter and a valuation sandbox (ValuationSandbox.tsx) where the reader can change assumptions and see the result.
4. Count every Pro conversion exactly once, whichever payment path recorded it, and track a funnel of five steps: landed, searched, opened a company, built a portfolio and went Pro.
5. Add Financial Shorts cards with real likes and comments. A summary may contain only numbers that appear in its headline or blurb; otherwise the card shows the headline alone.

RULES

- A language-model summary that adds a number its source did not contain is thrown away.
- News is matched to companies by name and symbol with rules for ambiguous names, and the market is decided from the news source.

CHECK A summary that adds a number its source did not contain is rejected.

EVIDENCE `insiderVolume.ts` `newsImpact.ts` `newsShorts.ts` `ValuationSandbox.tsx` `analytics.ts`

12a Community of theses

PHASE 12 · COMMUNITY AND BENCHMARKING

Build the community as a feed of investment theses with frozen evidence, not as a chat.

REQUIREMENTS

1. A post has a required title (120 characters), a body in light markdown (5,000 characters), up to 8 tagged companies and up to 6 attachments, with a 45-second cooldown per author.
2. Attachments are figures lifted from the app and frozen at posting with their source and date: a portfolio split, a chart, an image, key statistics, the score, statements, holders, insider activity, a fund or links.
3. Accept uploads as PNG, JPEG, GIF or WebP up to 5 MB. Identify them by their first bytes, refuse SVG, and store them under the SHA-256 of their contents.
4. Add follows (followed authors first, then recency), profiles at /community/u/[handle] with reserved handles, replies, likes stored as set membership, a notification inbox and a violet verification tick.
5. Add a draft editor that keeps an unsent post.

RULES

- Community identity is a one-way hash of the email, with no way back from a handle to an address.
- Handles are compared case-insensitively.

CHECK An SVG upload is refused even when it contains no script, and a handle cannot be duplicated by changing case.

EVIDENCE `communityPost.ts` `communityProfile.ts` `communityImage.ts` `follow.ts` `communityInbox.ts`

12b Honest portfolios and a design system

PHASE 12 · COMMUNITY AND BENCHMARKING

Make portfolio arithmetic correct across currencies, add benchmarking, and make the interface consistent.

REQUIREMENTS

1. Convert every holding into one display currency before computing any weight, total or gain. A row with a missing exchange rate shows as unavailable, never converted at one-to-one.
2. Compute weights in one function, portfolioBook.ts, so the allocation ring and the holdings table always agree.
3. Compare the book with the Nifty 50, S&P 500, FTSE 100, Nikkei 225 or Hang Seng as a time-weighted return, treating positions that enter part-way through as deposits (portfolioAlpha.ts).
4. Introduce a design system in System.tsx built only from semantic tokens, with the rule that information lives on the page and the default unit is a row, not a card.
5. Rebuild the landing page as a scroll-driven ascending price chart whose series comes from marketTape.ts. Make Financial Shorts a vertical deck that uses native scroll-snap.

RULES

- A rupee is never added to a dollar.
- An unknown exchange rate is null, never 1.

CHECK A rupee is never added to a dollar, and an unknown exchange rate is null, never 1.

EVIDENCE `portfolioFx.ts` `portfolioBook.ts` `portfolioAlpha.ts` `System.tsx` `marketTape.ts` `LandingAscent.tsx`

13 Nine-measure balance sheet and published portfolios

PHASE 13 · DEPTH AND GOVERNANCE

Deepen the analysis for non-financial companies, stop showing blanks where a figure can be derived, let users publish a portfolio safely, and document the compliance position.

REQUIREMENTS

1. Extend the industrial balance sheet from three measures to nine, including net debt to EBITDA, interest burden, the short-term share of debt, equity to assets, cash conversion and capital-expenditure cover.
2. When a ready-made ratio is missing, derive return on equity, return on assets, price to sales and PEG from the statements, and carry the derived figure's definition with it instead of showing a blank.
3. Let users publish a portfolio as weights only. Drop the amounts on the device before anything is sent, so ten times the money gives an identical publication, and keep the encrypted original out of reach of every reader's request.
4. Write down the compliance position: the terms of each data source, the disclaimers, and what the product does not claim.

RULES

- Portfolios and watchlists are stored encrypted (AES-256-GCM).
- A derived ratio always names its definition, so it is never mistaken for a reported one.

CHECK A published holding carries only a ticker and a weight, and a derived ratio always names its definition.

EVIDENCE `balanceSheet.ts` `yahooFundamentals.ts` `publicPortfolio.ts` `PortfolioSharing.tsx`